GI1 module: Théories des Langages & Compilation

Projet des automates

Membres du groupe :
- Ameziane Oumaima
- Lyamani Ismail
- Mohito Raihana

Remerciement :

Nous tenons à exprimer notre sincère gratitude à notre Professeur encadrant AL ACHHAB Mohammed, pour son accompagnement tout au long de ce projet. Grâce à ses conseils avisés, ses explications claires et ses remarques constructives, nous avons pu avancer avec méthode et rigueur. Ce projet a été pour nous une véritable opportunité d'apprentissage : il nous a permis d'approfondir nos connaissances, de découvrir de nouveaux concepts et de les mettre en pratique de manière concrète. Nous avons énormément appris et gagné en compétences, et pour cela, nous lui adressons nos plus vifs remerciements.

Table de contenu:

Remerciement :..... 1
I. Conception technique..... 3
1. Introduction: 3
2. Concepts théoriques: 3
3. Liste détaillée des fonctions élaborées : 3
II. Détails d'implémentation : Structures de données et mécanismes des fonctions 5
1. Structures de données :..... 5
2. Fonctions développées dans le cadre du projet : 6
III. Guide d'utilisation..... 39
1. Introduction..... 39
2. Installation et exécution..... 40
3. Format des fichiers d'Entrée..... 40
4. Interprétation des résultats..... 41

I. Conception technique

1. Introduction:

Les automates finis sont des modèles mathématiques utilisés pour représenter des systèmes de transitions d'états, notamment en théorie des langages formels. Ce projet implémente un ensemble d'opérations sur les automates finis déterministes (DFA) et non déterministes (NFA), permettant de manipuler des automates, de tester des mots, et de convertir des expressions régulières en automates.

2. Concepts théoriques:

- **Automate Fini** : Composé d'états, de transitions, d'un alphabet, d'états initiaux et d'états finaux.
- **Opérations Clés**:
    a. **Concaténation** : Combine deux automates A*A* et B*B* pour former A·B*A·B*, où un mot est accepté s'il traverse A*A* puis B*B*.
    b. **Union** : Combine A*A* et B*B* pour accepter les mots de A*A* **ou** B*B*.
    c. **Étoile de Kleene** : Répétition d'un automate A*A* zéro ou plusieurs fois (A∗*A*∗).
    d. **Déterminisation** : Convertit un NFA en DFA équivalent.
    e. **Élimination des ε-transitions** : Simplifie un automate en supprimant les transitions spontanées.
    f. **Intersection** : Accepte les mots reconnus par deux automates simultanément.
    g. **Minimisation** : Transforme un **DFA** en un automate **minimal** (avec le nombre minimal d'états) reconnaissant le même langage.

3. Liste détaillée des fonctions élaborées :

a. Gestion des états et transitions :

- parc_etat_init(automate* aut, int Num_etat)
  Marque un état comme initial.
- parc_etat_fin(automate* aut, int Num_etat)
  Marque un état comme final.

b. Lecture/Écriture d'automates:

- lire_automate_dot(const char* nomFichier, automate* automate)
  Lit un automate depuis un fichier .dot.
- creer_dot(automate* automate, const char* nom_fichier)
  Génère un fichier .dot représentant l'automate.

c. Affichage et libération mémoire:

- afficher_automate(automate *automate)
  Affiche les états, transitions et alphabet de l'automate.
- liberer_automate(automate *automate)
  Libère la mémoire allouée pour l'automate.

d. Validation de mots:

- explorer(automate *aut, int etat_actuel, const char *mot, int index)
  Fonction récursive pour explorer les chemins possibles d'un mot.
- verifierMot(automate *automate, const char *mot)
  Vérifie si un mot est accepté par l'automate.

- tester_mot(automate *automate)
  Demande un mot à l'utilisateur et teste son acceptation.
- tester_mots_fichier(automate *automate, const char *nom_fichier)
  Teste tous les mots d'un fichier texte.

e. Opérations sur les automates:

- concatener_automates(automate* aut1, automate* aut2)
  Concatène deux automates (A.B).
- union_automate(automate* aut1, automate* aut2)
  Crée l'union de deux automates (A|B).
- plusAutomate(automate* aut)
  Applique l'opérateur '+' (au moins une répétition).
- etoileAutomate(automate *aut)
  Applique l'opérateur '*' (répétition zéro ou plus).
- Intersection(automate* A3, automate* A4, const char* nom_fichier_destination)
  Crée l'intersection de deux automates.

f. Utilitaires:

- Etats_Plus_Grand_Transition(automate* automat, int* nb_etats_max)
  Retourne les états avec le plus de transitions sortantes.
- etat_etiquette(automate *automat, char etiquette, int *nb_etat)
  Liste les états ayant une transition avec une étiquette donnée.
- calculer_etats_init(automate* aut)
  Compte les états initiaux.
- calculer_etats_fin(automate* aut)
  Compte les états finaux.

g. Conversion d'expressions régulières:

- creer_automate_symbole(char c)
  Crée un automate élémentaire pour un symbole.
- regexToAutomateRecursive(const char* regex, int* index)
  Convertit récursivement une expression régulière en automate.
- regexToAutomate(const char* regex)
  Point d'entrée pour la conversion d'une expression régulière.

h. Optimisation et transformations:

- calculer_fermeture_epsilon(automate *aut, int etat, bool *fermeture)
  Calcule la fermeture ε d'un état.
- marquer_etats_accessibles(automate* aut, int etat, bool* accessibles)
  Marque les états accessibles depuis un état donné.
- supprimer_etats_non_accessibles(automate* aut)
  Supprime les états non accessibles.
- supprimer_epsilon_transitions(automate aut)
  Élimine les transitions ε.
- determiniser_automate(automate* afn)
  Convertit un NFA en DFA.
- renverser_automate(automate* aut)
  Inverse les transitions de l'automate.

i. Interface utilisateur:

- menu()
  Affiche un menu interactif pour manipuler les automates.
- main()
  Point d'entrée du programme, lance le menu.

j. Fonctions annexes:

- compare_ints(const void* a, const void* b)
  Compare deux entiers pour le tri.
- contient_etat(int* ensemble, int taille, int etat)
  Vérifie si un état est dans un ensemble.
- ensembles_egaux(int* a, int taille_a, int* b, int taille_b)
  Compare deux ensembles d'états.
- trouver_ou_ajouter_ensemble(int*** ensembles, int* nb_ensembles, int* taille_ensemble, int* nouvel_ensemble, int taille_nouvel)
  Gestion des ensembles d'états pour la déterminisation.

II. Détails d'implémentation : Structures de données et mécanismes des fonctions

1. Structures de données :

Les structures de données définies dans notre projet (etats, transitions, automate) sont conçues pour modéliser un **automate fini.**

a. Structure **etats** :

```
typedef struct {
 int num_etat;
    bool is_final;
    bool is_initial;
} etats;
```

Cette structure sert à la représentation d'un état dans l'automate :

- **num_etat :** Permet d'identifier l'état de manière unique (ex : 0, 1).
- **is_final :** Détermine si l'état est un état final.
- **is_initial :** Détermine si l'état est un point de départ pour l'exécution de l'automate.

➔ Les états initiaux et finaux sont critiques pour définir le comportement de l'automate. Pour cette raison, nous avons envisagé les intégrer parmi les caractéristiques d'un état.

b. Structure **transitions** :

```
// Définition de la structure pour les transitions
typedef struct {
    etats etat_depart;
    etats etat_arrive;
    char etiquette;
} transitions;
```

Cette structure représente une transition entre deux états.

- **etat_depart** et **etat_arrive :** Les états reliés par la transition.
- **etiquette :** Le symbole de l'alphabet qui déclenche cette transition (ex : a, b, ou x pour ε-transitions).

c. Structure **automate** :

```
// Définition de la structure pour l'automate
typedef struct {
    etats *etats;
    int nb_etat;
    transitions* transition;
    int nb_transition;
    char *alphabet;
    int nbr_symbole;
} automate;
```

Cette structure centrale représente l'automate entier avec tous ses composants.

- **etats et transition :** Tableaux dynamiques pour stocker les états et les transitions (assure la flexibilité pour les opérations de modification).
- **alphabet :** Liste des symboles valides pour les transitions .
- **nbr_symbole** : Permet de gérer dynamiquement la taille de l'alphabet.

2. Fonctions développées dans le cadre du projet :

a. Gestion des états et transitions :

Ces fonctions servent à marquer un état spécifique d'un automate comme état initial ou état final. Elles sont utilisées lors de la construction ou de la modification d'un automate**.**

- **`parc_etat_init` :** Parcourt tous les états de l'automate. Si le numéro d'état (`Num_etat`) correspond à un état trouvé dans le tableau, son drapeau `is_initial` est défini à `true`. Elle permet de marquer **plusieurs états comme initiaux.**
- **`parc_etat_fin` :** Similaire à **parc_etat_init**, mais marque l'état comme final (is_final = true). Elle permet de définir **plusieurs états finals.**

```
// Fonction pour marquer un état comme initial
void parc_etat_init(automate* aut, int Num_etat){
    for (int i=0; i<aut->nb_etat; i++){
        if (aut->etats[i].num_etat==Num_etat){
            aut->etats[i].is_initial=true;
        }
    }
}

// Fonction pour marquer un état comme final
void parc_etat_fin(automate* aut, int Num_etat){
    for (int i=0; i<aut->nb_etat; i++){
        if (aut->etats[i].num_etat==Num_etat){
            aut->etats[i].is_final=true;
        }
    }
}
```

b. Lecture/Écriture d'automates:

✓ **But de la fonction**

Charger un automate à partir d'un fichier .dot :

- Identifier les **états**, les **transitions**, les **symboles de l'alphabet**, les **états initiaux** et les **états finaux**.
- Remplir une structure automate avec ces informations.

Cette fonction fonctionne comme suit :

- **Ouverture du fichier .dot** en lecture.
- **Premier parcours du fichier** :
  - Compter le **nombre d'états**
  - Compter le **nombre de transitions**
- **Allocation dynamique** de la mémoire pour :
  - La liste des **états**
  - La liste des **transitions**
  - L'**alphabet** (max 26 caractères)
- **Deuxième parcours du fichier** pour remplir :
  - Les **transitions** :
    - Extraction de l'état de départ, d'arrivée et du ou des symboles
    - Ajout des symboles à l'alphabet s'ils sont nouveaux
  - Les **états** :
    - Lecture des numéros des états déclarés dans le fichier
    - Initialisation avec is_initial = false et is_final = false
  - L'**état initial** :
    - Détecté à partir du mot-clé start
    - Enregistré avec la fonction parc_etat_init
  - Les **états finaux** :
    - Détectés à partir du mot-clé final
    - Marqués avec la fonction parc_etat_fin
- **Fermeture du fichier** après traitement

⇒ **Pourquoi deux parcours du fichier ?**

- En langage C, il faut **connaître la taille exacte** des données avant d'allouer la mémoire (via malloc).
- Le **1er parcours** sert à :
  - **Compter le nombre d'états et de transitions**
  - Préparer la **mémoire nécessaire** pour les stocker
- Le **2e parcours** sert à :
  - **Lire les vraies données** (états, transitions, alphabet…)
  - **Remplir les structures** de l'automate (tableaux dynamiques)

➔ Cette méthode est :

- **Plus sûre** (évite les erreurs de mémoire)
- **Plus efficace** (mieux que de réallouer à chaque ajout)

```
// Fonction pour lire un automate à partir d'un fichier .dot
void lire_automate_dot(const char* nomFichier, automate* automate) {
    FILE *fichier = fopen(nomFichier, "r");
    if (fichier == NULL) {
        printf("Erreur : impossible d'ouvrir le fichier %s !\n", nomFichier);
        exit(1);
```

```
    }

    char ligne[256];
    int nb_etats = 0;
    int nb_transition = 0;
    int i=0;
    // Compter le nombre d'états et de transitions
    while (fgets(ligne, sizeof(ligne), fichier)) {
        if (strstr(ligne, "shape")) {
            char* n= ligne + i;
            nb_etats++;
            sscanf(n, "    node [shape = circle]; %*d%n",&i);
            n= ligne + i;
            while(n[0]!=';'){
                nb_etats++;
                sscanf(n,", %*d%n",&i);
                n= n + i;
            }
        }
        if (strstr(ligne, "->")) {
            nb_transition++;
        }
    }

    // Allocation dynamique de la mémoire
    automate->etats = malloc(nb_etats * sizeof(etats));
    automate->transition = malloc(nb_transition * sizeof(transitions));
    automate->alphabet = malloc(26 * sizeof(char));
    memset(automate->alphabet, 0, 26); // Initialisation

    automate->nb_etat = 0;
    automate->nb_transition = 0;
    automate->nbr_symbole = 0;

    rewind(fichier); // Revenir au début du fichier
    while (fgets(ligne, sizeof(ligne), fichier)) {
        if (strstr(ligne, "label")) {
            transitions transition;
            char symbole[10];
            int etat_depart, etat_arrive;
            sscanf(ligne, "%d -> %d [label=\"%[^\"]", &etat_depart, &etat_arrive,
symbole);
            transition.etat_depart = automate->etats[etat_depart];
            transition.etat_arrive = automate->etats[etat_arrive];
            transition.etiquette = symbole[0];
```

```
            automate->transition[automate->nb_transition++] = transition;
            // Ajouter à l'alphabet si non existant
            if (strchr(automate->alphabet, symbole[0]) == NULL) {
                automate->alphabet[automate->nbr_symbole++] = symbole[0];
                automate->alphabet[automate->nbr_symbole] = '\0'; // Marquer la
fin
            }
            // Lire les autres symboles séparés par des virgules
            char *symbole_ptr = strchr(ligne, ',');
            while (symbole_ptr != NULL) {
                symbole_ptr++;
                if (sscanf(symbole_ptr, "%c", &transition.etiquette) == 1) {
                    automate->transition[automate->nb_transition++] = transition;
                    if (strchr(automate->alphabet, transition.etiquette) == NULL)
{
                        automate->alphabet[automate->nbr_symbole++] =
transition.etiquette;
                        automate->alphabet[automate->nbr_symbole] = '\0'; //
Marquer la fin
                    }
                }
                symbole_ptr = strchr(symbole_ptr, ',');
            }
        }
        else if (strstr(ligne, "[shape = circle")) {
            etats etat;
            etat.is_final = false;
            etat.is_initial = false;
            int index=0;
            char* note= ligne + index;
            sscanf(note, "    node [shape = circle]; %d%n", &etat.num_etat,
&index);
            automate->etats[automate->nb_etat++] = etat;
            note= ligne + index;
            while(note[0]!=';'){
                etat.is_final = false;
                etat.is_initial = false;
                sscanf(note,", %d%n",&etat.num_etat,&index);
                automate->etats[automate->nb_etat++] = etat;
                note= note + index;
            }
        } else if (strstr(ligne, "[shape = point width = 0]; start")){
            fgets(ligne, sizeof(ligne), fichier);
            int n;
            sscanf(ligne, "    %*s -> %d", &n);
```

```
            parc_etat_init(automate, n);
        } else if (strstr(ligne, "[shape = point width = 0]; final")){
            fgets(ligne, sizeof(ligne), fichier);
            int n;
            sscanf(ligne, "    %d -> %*s", &n);
            parc_etat_fin(automate, n);
        }
    }
    fclose(fichier);
}
```

✓ **But de fonction :**

Elle génère un fichier .dot contenant la représentation graphique de l'automate.
Ce fichier peut ensuite être visualisé avec Graphviz pour voir l'automate sous forme de graphe.

⇒ **Mécanisme**

1. **Ouvre un fichier en écriture** (avec extension `.dot`)
2. Écrit les **déclarations de base** du graphe en langage DOT :
   - Début du graphe dirigé (`digraph G {`)
   - Définition des nœuds (états) de forme circulaire
3. Écrit tous les **états** de l'automate comme des nœuds
4. Écrit les **transitions** :
   - Pour chaque transition : ligne du type `état1 -> état2 [label="a"]`
   - Où `a` est le symbole lu
5. Gère les **états initiaux** :
   - Ajoute un nœud invisible nommé `start` et une flèche vers l'état initial
6. Gère les **états finaux** :
   - Ajoute une flèche de l'état final vers un nœud invisible `final`
7. Termine le fichier DOT avec une accolade fermante `}`

```
void creer_dot(automate* automate, const char* nom_fichier) {
    FILE* fichier = fopen(nom_fichier, "w");
    if (fichier == NULL) {
        printf("Erreur : impossible d'ouvrir le fichier %s !\n", nom_fichier);
        exit(1);
    }
    fprintf(fichier, "digraph G {\n");
    fprintf(fichier, "    rankdir = LR;\n");
    fprintf(fichier, "    node [shape = circle];");
    for (int i = 0; i < automate->nb_etat; i++) {
        if (i == 0) {
            fprintf(fichier, " %d", automate->etats[i].num_etat);
        } else if (i == automate->nb_etat - 1) {
            fprintf(fichier, ", %d;\n", automate->etats[i].num_etat);
        } else {
```

```
            fprintf(fichier, ", %d", automate->etats[i].num_etat);
        }
    }
    int ni = 1;
    int nf = 1;
    for (int i = 0; i < automate->nb_etat; i++) {
        if (automate->etats[i].is_initial && automate->etats[i].is_final) {
            fprintf(fichier, "    node [shape = point width = 0]; start%d;\n",
ni);
            fprintf(fichier, "    start%d -> %d;\n", ni, automate-
>etats[i].num_etat);
            ni++;
            fprintf(fichier, "    node [shape = point width = 0]; final%d;\n",
nf);
            fprintf(fichier, "    %d -> final%d;\n", automate->etats[i].num_etat,
nf);
            nf++;
        } else if (automate->etats[i].is_initial && !automate->etats[i].is_final)
{
            fprintf(fichier, "    node [shape = point width = 0]; start%d;\n",
ni);
            fprintf(fichier, "    start%d -> %d;\n", ni, automate-
>etats[i].num_etat);
            ni++;
        } else if (automate->etats[i].is_final && !automate->etats[i].is_initial)
{
            fprintf(fichier, "    node [shape = point width = 0]; final%d;\n",
nf);
            fprintf(fichier, "    %d -> final%d;\n", automate->etats[i].num_etat,
nf);
            nf++;
        }
    }
    for (int i = 0; i < automate->nb_transition; i++) {
        fprintf(fichier, "    %d -> %d [label=\"%c\"];\n", automate-
>transition[i].etat_depart.num_etat, automate-
>transition[i].etat_arrive.num_etat, automate->transition[i].etiquette);
    }
    fprintf(fichier, "}\n");
    fclose(fichier);
}
```

c. Affichage et libération mémoire:

✓ **But de fonction :**

Elle affiche **toutes les informations importantes** d'un automate déjà chargé en mémoire :

- Le **nombre d'états**
- Le détail de chaque **état** (avec indication s'il est initial ou final)
- Le **nombre de transitions**
- Le détail de chaque **transition**
- L'**alphabet** utilisé (ensemble des symboles lus)

⇒ **Mécanisme :**

- Vérifie si l'automate est vide (NULL) :
  - Si oui, affiche un message d'erreur et quitte la fonction
- Affiche le **nombre total d'états**
- Parcourt tous les états pour :
  - Afficher le **numéro de l'état**
  - Indiquer s'il est **initial** ou **final**
- Affiche le **nombre total de transitions**
- Parcourt toutes les transitions pour :
  - Afficher sous la forme état_depart --symbole--> état_arrive
- Affiche les **symboles de l'alphabet** entre accolades {}, séparés par des virgules

```
// Fonction pour afficher les informations de l'automate
void afficher_automate(automate *automate) {
    if (!automate) {
        printf("L'automate est vide.\n");
        return;
    }

    printf("Nombre d'etats : %d\n", automate->nb_etat);
    printf("Etats :\n");

    for (int i = 0; i < automate->nb_etat; i++) {
        printf("Etat %d, Initial: %s, Final: %s\n",
               automate->etats[i].num_etat,
               automate->etats[i].is_initial ? "Oui" : "Non",
               automate->etats[i].is_final ? "Oui" : "Non");
    }
    printf("\nNombre de transitions : %d\n", automate->nb_transition);
    for (int i = 0; i < automate->nb_transition; i++) {
        printf("%d --%c--> %d\n", automate->transition[i].etat_depart.num_etat,
               automate->transition[i].etiquette,
               automate->transition[i].etat_arrive.num_etat);
    }
    printf("Les alphabets de l'automate sont : {");
    for (int i = 0; i < automate->nbr_symbole; i++) {
        if (i == automate->nbr_symbole - 1) {
            printf("%c", automate->alphabet[i]);
            continue;
        }else{
```

```
            printf("%c,", automate->alphabet[i]);
        }
    }
    printf("}\n");
}
```

✓ **But de la fonction :**

Libérer toute la mémoire dynamique allouée à un automate (états, transitions, alphabet) pour éviter les **fuites mémoire** une fois que l'automate n'est plus utilisé.

```
// Fonction pour libérer la mémoire allouée pour l'automate
void liberer_automate(automate *automate) {
    free(automate->etats);
    free(automate->transition);
    free(automate->alphabet);
}
```

d. Validation de mots:

✓ **But de la fonction :**

Explorer récursivement les états de l'automate pour déterminer si un mot est accepté en partant d'un état donné, en suivant les transitions qui correspondent aux symboles du mot.

⇒ **Mécanisme :**

1. Vérifier si le mot est complètement consommé.
   - Si oui, vérifier si l'état actuel est un état final.
2. Si le mot n'est pas encore complètement consommé :
   - Prendre le prochain symbole du mot.
   - Explorer les transitions possibles de l'état actuel correspondant à ce symbole.
   - Recommencer l'exploration sur l'état suivant via récursion.
3. Retourner true si le mot est accepté (nous arrivons à un état final), sinon false.

```
bool explorer(automate *automate, int etat_actuel, const char *mot, int index) {
    if (mot[index] == '\0') {
        // Si on a consommé tout le mot, vérifier si on est dans un état final
        for (int i = 0; i < automate->nb_etat; i++) {
            if (automate->etats[i].num_etat == etat_actuel && automate-
>etats[i].is_final) {
                return true; // Le mot est accepté
            }
        }
        return false;
    }

    char symbole = mot[index];
    bool accepte = false;

    // Parcourir toutes les transitions pour trouver les chemins possibles
```

```
    for (int i = 0; i < automate->nb_transition; i++) {
        if (automate->transition[i].etat_depart.num_etat == etat_actuel &&
automate->transition[i].etiquette == symbole) {
            // Explorer récursivement l'état suivant
            if (explorer(automate, automate->transition[i].etat_arrive.num_etat,
mot, index + 1)) {
                accepte = true;
                break; // Si un chemin accepte le mot, inutile de chercher plus
            }
        }
    }

    return accepte;
}
```

✓ **But de la fonction :**

Vérifier si un mot est accepté par l'automate en commençant l'exploration depuis l'état initial de l'automate.

⇒ **Mécanisme :**

1. Trouver l'état initial de l'automate.
2. Lancer l'exploration à partir de cet état en appelant la fonction explorer.
3. Si le mot est accepté (la fonction explorer retourne true), retourner true. Sinon, retourner false.

```
// Vérification principale du mot
bool verifierMot(automate *automate, const char *mot) {
    // Trouver l'état initial
    for (int i = 0; i < automate->nb_etat; i++) {
        if (automate->etats[i].is_initial) {
            // Lancer l'exploration à partir de cet état
            if (explorer(automate, automate->etats[i].num_etat, mot, 0)) {
                return true;
            }
        }
    }

    return false;
}
```

✓ **But de la fonction :**

Permettre à l'utilisateur de tester un mot en l'explorant dans l'automate et afficher si ce mot est accepté ou rejeté.

⇒ **Mécanisme :**

1. Demander à l'utilisateur d'entrer un mot à tester.
2. Vérifier si ce mot est accepté en appelant la fonction verifierMot.

3. Afficher si le mot est accepté ou rejeté selon le résultat de la vérification.

```c
// Fonction pour demander à l'utilisateur un mot et le tester
void tester_mot(automate *automate) {
    char mot[100];

    printf("Entrez un mot a tester : ");
    scanf("%s", mot);

    if (verifierMot(automate, mot)) {
        printf("Le mot \"%s\" est accepte par l'automate.\n", mot);
    } else {
        printf("Le mot \"%s\" est rejete.\n", mot);
    }
}
```

✓ **But de la fonction :**

Tester une liste de mots extraits d'un fichier texte et afficher si chaque mot est accepté ou rejeté par l'automate.

⇒ **Mécanisme :**

1. Ouvrir le fichier contenant une liste de mots.
2. Lire chaque mot dans le fichier.
3. Vérifier si chaque mot est accepté par l'automate en appelant verifierMot.
4. Afficher les résultats pour chaque mot : accepté ou rejeté.

```c
// Fonction pour tester les mots dans un fichier texte
void tester_mots_fichier(automate *automate, const char *nom_fichier) {
    FILE *fichier = fopen(nom_fichier, "r");
    if (fichier == NULL) {
        printf("Erreur : impossible d'ouvrir le fichier %s !\n", nom_fichier);
        exit(1);
    }

    char mot[256];
    int index = 0;
    char c;

    while ((c = fgetc(fichier)) != EOF) {
        if (isspace(c)) {
            if (index > 0) {
                mot[index] = '\0';
                if (verifierMot(automate, mot)) {
                    printf("Le mot \"%s\" est accepte par l'automate.\n", mot);
                } else {
                    printf("Le mot \"%s\" est rejete par l'automate.\n", mot);
```

```
                }
                index = 0;
            }
        } else {
            mot[index++] = c;
        }
    }

    // Vérifier le dernier mot s'il y en a un
    if (index > 0) {
        mot[index] = '\0';
        if (verifierMot(automate, mot)) {
            printf("Le mot \"%s\" est accepte par l'automate.\n", mot);
        } else {
            printf("Le mot \"%s\" est rejete par l'automate.\n", mot);
        }
    }

    fclose(fichier);
}
```

e. Opérations sur les automates:

✓ **But de la fonction :**
Combiner deux automates *A* et *B* pour former *A·B*, où un mot est accepté s'il traverse *A* puis *B*. Les états finaux de *A* sont connectés aux états initiaux de *B* via des transitions epsilon ('x').

⇒ **Mécanisme :**

1. **Création d'un nouvel automate** avec un nombre d'états égal à la somme des états de *A* et *B*.
2. **Copie des transitions** de *A* et *B* dans le nouvel automate.
3. **Ajout de transitions epsilon** entre les états finaux de *A* et les états initiaux de *B*.
4. **Mise à jour de l'alphabet** en fusionnant les symboles de *A* et *B*, et en ajoutant 'x' pour les transitions epsilon.

```
automate* concatener_automates(automate* aut1, automate* aut2) {
    automate* new_automate = malloc(sizeof(automate));
    new_automate->nb_etat = aut1->nb_etat + aut2->nb_etat;
    // ... (Copie des états et transitions)
    for (int i = 0; i < aut1->nb_etat; i++) {
        if (aut1->etats[i].is_final) {
            for (int j = 0; j < aut2->nb_etat; j++) {
                if (aut2->etats[j].is_initial) {
                    // Ajouter une transition epsilon
                }
            }
```

```
        }
    }
    return new_automate;
}
```

But de la fonction
Le but de cette fonction est de créer un nouvel automate qui représente l'union de deux automates donnés . Cela signifie que le nouvel automate acceptera une chaîne si elle est acceptée par au moins un des deux automates d'entrée.

Mécanisme de la fonction

1. **Création du nouvel automate** :
    - Allouer un automate (new_automate) qui contiendra les états et transitions des deux automates d'entrée, ainsi que deux nouveaux états : un nouvel état initial et un nouvel état final.
2. **Ajout des états** :
    - Ajouter un nouvel état initial et un nouvel état final au nouvel automate.
    - Copier les états des deux automates d'entrée dans le nouvel automate, en ajustant leurs numéros pour éviter les conflits.
3. **Ajout des transitions** :
    - Copier les transitions des deux automates d'entrée dans le nouvel automate, en ajustant les numéros des états pour correspondre à leur nouvelle position dans le nouvel automate.
    - Ajouter des transitions du nouvel état initial vers les états initiaux des deux automates d'entrée.
    - Ajouter des transitions des états finaux des deux automates d'entrée vers le nouvel état final.
4. **Gestion de l'alphabet** :
    - Fusionner les alphabets des deux automates d'entrée pour créer l'alphabet du nouvel automate.
5. **Retour du nouvel automate :**
    - Retourner le nouvel automate qui accepte l'union des deux langages.

```
// Fonction pour faire l'union de deux automates
automate* union_automate(automate* aut1, automate* aut2){
    automate* new_automate = malloc(sizeof(automate));
    int nbt = aut1->nb_transition + aut2->nb_transition +
calculer_etats_fin(aut1) + calculer_etats_fin(aut2) + calculer_etats_init(aut1) +
calculer_etats_init(aut2);
    new_automate->nb_etat = aut1->nb_etat + aut2->nb_etat + 2;
    new_automate->nb_transition = aut1->nb_transition + aut2->nb_transition;
    new_automate->etats = malloc(new_automate->nb_etat * sizeof(etats));
    new_automate->transition = malloc(nbt * sizeof(transitions));
    new_automate->alphabet = malloc(26 * sizeof(char));
    memset(new_automate->alphabet, 0, 26); // Initialisation
```

```
    etats etat_init;
    etats etat_fin;
    etat_init.is_initial = true;
    etat_init.is_final = false;
    etat_init.num_etat = 0;
    etat_fin.is_initial = false;
    etat_fin.is_final = true;
    etat_fin.num_etat = new_automate->nb_etat-1;
    new_automate->etats[0] = etat_init;
    new_automate->etats[new_automate->nb_etat - 1] = etat_fin;
    for(int i = 1; i < new_automate->nb_etat - 1; i++){
        new_automate->etats[i].num_etat = i;
        new_automate->etats[i].is_initial = false;
        new_automate->etats[i].is_final = false;
    }
    int compteur_transitions=0;
    for (int i = 0; i < aut1->nb_transition; i++){
        new_automate->transition[compteur_transitions]=aut1->transition[i];
        new_automate->transition[compteur_transitions].etat_depart.num_etat += 1;
        new_automate->transition[compteur_transitions].etat_arrive.num_etat += 1;
        compteur_transitions++;
    }
    for (int i = 0; i < aut2->nb_transition; i++){
        new_automate->transition[compteur_transitions]=aut2->transition[i];
        new_automate->transition[compteur_transitions].etat_depart.num_etat +=
1+aut1->nb_etat;
        new_automate->transition[compteur_transitions].etat_arrive.num_etat +=
1+aut1->nb_etat;
        compteur_transitions++;
    }
    int compteur_etat = 1;
    for (int i = 0; i < aut1->nb_etat; i++){
        if (aut1->etats[i].is_initial)
        {
            transitions new_transition;
            new_transition.etat_depart = new_automate->etats[0];
            new_transition.etat_arrive = new_automate->etats[compteur_etat];
            new_transition.etiquette = 'x';
            new_automate->transition[new_automate->nb_transition++] =
new_transition;
        }
        compteur_etat++;
    }
    for (int i = 0; i < aut2->nb_etat; i++){
        if (aut2->etats[i].is_initial)
```

```
        {
            transitions new_transition;
            new_transition.etat_depart = new_automate->etats[0];
            new_transition.etat_arrive = new_automate->etats[compteur_etat];
            new_transition.etiquette = 'x';
            new_automate->transition[new_automate->nb_transition++] =
new_transition;
        }
        compteur_etat++;
    }
    compteur_etat = 1;
    for (int  i = 0; i < aut1->nb_etat; i++){
        if(aut1->etats[i].is_final){
            transitions new_transition;
            new_transition.etat_arrive = new_automate->etats[(new_automate-
>nb_etat - 1)];
            new_transition.etat_depart = new_automate->etats[compteur_etat];
            new_transition.etiquette = 'x';
            new_automate->transition[new_automate->nb_transition++] =
new_transition;
        }
        compteur_etat++;
    }
    for (int  i = 0; i < aut2->nb_etat; i++){
        if(aut2->etats[i].is_final){
            transitions new_transition;
            new_transition.etat_arrive = new_automate->etats[new_automate-
>nb_etat - 1];
            new_transition.etat_depart = new_automate->etats[compteur_etat];
            new_transition.etiquette = 'x';
            new_automate->transition[new_automate->nb_transition++] =
new_transition;
        }
        compteur_etat++;
    }
    int k = 0;
    for (int i = 0; i < aut1->nbr_symbole; i++) {
        new_automate->alphabet[k++] = aut1->alphabet[i];
    }
    for (int i = 0; i < aut2->nbr_symbole; i++) {
        if (strchr(new_automate->alphabet, aut2->alphabet[i]) == NULL) {
            new_automate->alphabet[k++] = aut2->alphabet[i];
        }
    }
    new_automate->alphabet[k] = 'x';
```

```
    new_automate->nbr_symbole = k;
    return new_automate;
}
```

✓ **But de la fonction**

Le but de la fonction plusAutomate est de transformer un automate donné (aut) en un nouvel automate qui accepte le langage correspondant à l'itération positive (L+) du langage accepté par l'automate d'entrée. Cela signifie que le nouvel automate acceptera toutes les chaînes qui peuvent être générées par une ou plusieurs répétitions des chaînes acceptées par l'automate d'origine.

⇒ **Mécanisme de la fonction**

1. **Création du nouvel automate** :
   - Allouer un nouvel automate (new_automate) avec le même nombre d'états que l'automate d'entrée.
   - Calculer le nombre total de transitions pour inclure les transitions supplémentaires nécessaires pour réaliser l'itération positive.
2. **Copie des états** :
   - Copier tous les états de l'automate d'entrée dans le nouvel automate.
3. **Copie des transitions existantes** :
   - Copier toutes les transitions de l'automate d'entrée dans le nouvel automate.
4. **Ajout des transitions pour l'itération positive :**
   - Ajouter des transitions des états finaux de l'automate d'entrée vers les états initiaux. Cela permet de boucler sur l'automate pour accepter plusieurs répétitions des chaînes.
5. **Gestion de l'alphabet :**
   - Copier l'alphabet de l'automate d'entrée dans le nouvel automate.
   - Ajouter un symbole spécial 'x' à l'alphabet pour représenter les transitions ajoutées.
6. **Retour du nouvel automate :**
   - Retourner le nouvel automate qui accepte le langage correspondant à l'itération positive.

```
automate* plusAutomate(automate* aut){
    automate* new_automate = malloc(sizeof(automate));
    new_automate->nb_etat = aut->nb_etat ;
    new_automate->nb_transition = aut->nb_transition+ calculer_etats_fin(aut) *
calculer_etats_init(aut);
    new_automate->etats = malloc(new_automate->nb_etat * sizeof(etats));
    new_automate->transition = malloc(new_automate->nb_transition *
sizeof(transitions));
    new_automate->alphabet = malloc(26 * sizeof(char));
    memset(new_automate->alphabet, 0, 26); // Initialisation

    for(int i=0; i<new_automate->nb_etat; i++){
        new_automate->etats[i] = aut->etats[i];
```

```
    }
    int compteur_transitions=0;
    for (int i = 0; i < aut->nb_transition; i++){
        new_automate->transition[compteur_transitions]=aut->transition[i];
        compteur_transitions++;
    }
    for (int i = 0; i < aut->nb_etat; i++){
        if (aut->etats[i].is_initial){
            for (int j = 0; j < aut->nb_etat; j++){
                new_automate-
>transition[compteur_transitions]=(transitions){new_automate->etats[j],
new_automate->etats[i], 'x'};
            }
        }
    }

    int k = 0;
    for (int i = 0; i < aut->nbr_symbole; i++) {
        new_automate->alphabet[k++] = aut->alphabet[i];
    }
    for (int i = 0; i < aut->nbr_symbole; i++) {
        if (strchr(new_automate->alphabet, aut->alphabet[i]) == NULL) {
            new_automate->alphabet[k++] = aut->alphabet[i];
        }
    }
    new_automate->alphabet[k] = 'x';
    new_automate->nbr_symbole = k;
    return new_automate;
}
```

✓ **But de la fonction :**

Le but de la fonction etoileAutomate est de transformer un automate donné en un nouvel automate qui accepte le langage étoile (L*) de celui-ci. Cela signifie que le nouvel automate acceptera zéro ou plusieurs répétitions des chaînes reconnues par l'automate original.

⇒ **Mécanisme de la function :**

1. **Transformation initiale** :
   - La fonction commence par appliquer plusAutomate sur l'automate original pour obtenir un automate reconnaissant L+ (au moins une répétition).
2. **Création du nouvel automate** :
   - Allocation d'un nouveau automate avec 2 états supplémentaires :
     - Un nouvel état initial.
     - Un nouvel état final.

- Initialisation de ces nouveaux états.

3. **Copie et ajustement des anciens éléments** :
   - Les états de l'automate modifié (L+) sont copiés au centre du nouvel automate.
   - Les transitions sont également copiées, avec une mise à jour des numéros d'états (décalage de +1).
4. **Ajout des transitions epsilon** (`'x'`) :
   - Du nouvel état initial vers l'ancien initial.
   - Du nouvel état initial directement vers le nouvel état final (permettant l'acceptation de la chaîne vide).
   - De l'ancien état final vers le nouvel état final.
5. **Mise à jour de l'alphabet** :
   - L'alphabet est copié depuis l'automate d'origine.
   - Le symbole spécial `'x'` est ajouté pour représenter les transitions epsilon.
6. **Retour** :
   - Le nouvel automate construit est retourné. Il accepte toutes les chaînes du langage L*

```
//pour etoile d'un automate *
automate* etoileAutomate(automate *aut){
    aut=plusAutomate(aut);
    automate* new_automate = malloc(sizeof(automate));
    new_automate->nb_etat = aut->nb_etat + 2;
    new_automate->nb_transition = aut->nb_transition + 3;
    new_automate->etats = malloc(new_automate->nb_etat * sizeof(etats));
    new_automate->transition = malloc(new_automate->nb_transition *
sizeof(transitions));
    new_automate->alphabet = malloc(26 * sizeof(char));
    memset(new_automate->alphabet, 0, 26); // Initialisation
    new_automate->etats[0] = (etats){0, false, true};
    new_automate->etats[new_automate->nb_etat - 1] = (etats){new_automate-
>nb_etat - 1, true, false};
    for(int i=1; i<new_automate->nb_etat-1; i++){
        new_automate->etats[i] = (etats){i, false, false};
    }
    int compteur_transitions=0;
    for (int i = 0; i < aut->nb_transition; i++){
        new_automate->transition[compteur_transitions]=aut->transition[i];
        new_automate->transition[compteur_transitions].etat_depart.num_etat += 1;
        new_automate->transition[compteur_transitions].etat_arrive.num_etat += 1;
        compteur_transitions++;
    }
    new_automate->transition[compteur_transitions]= (transitions){new_automate-
>etats[0], new_automate->etats[new_automate->nb_etat - 1], 'x'};
    compteur_transitions++;
    new_automate->transition[compteur_transitions]=(transitions){new_automate-
>etats[0], new_automate->etats[1], 'x'};
```

```
    compteur_transitions++;
    new_automate->transition[compteur_transitions]=(transitions){new_automate-
>etats[new_automate->nb_etat-2], new_automate->etats[new_automate->nb_etat-1],
'x'};
    int k = 0;
    for (int i = 0; i < aut->nbr_symbole; i++) {
        new_automate->alphabet[k++] = aut->alphabet[i];
    }
    for (int i = 0; i < aut->nbr_symbole; i++) {
        if (strchr(new_automate->alphabet, aut->alphabet[i]) == NULL) {
            new_automate->alphabet[k++] = aut->alphabet[i];
        }
    }
    new_automate->alphabet[k] = 'x';
    new_automate->nbr_symbole = k;
    return new_automate;
}
```

✓ **But de la fonction :**

Le but de la fonction creer_automate_symbole est de construire un automate simple qui reconnaît un seul symbole donné en entrée. Cet automate accepte uniquement la chaîne composée du caractère c, et rien d'autre.

⇒ **Mécanisme de la fonction**

1. **Création de la structure de base** :
   - L'automate est initialisé avec deux états :
     - Un état initial (non final).
     - Un état final (non initial).
2. **Ajout de la transition** :
   - Une seule transition est ajoutée entre l'état initial et l'état final avec le symbole c.
3. **Définition de l'alphabet** :
   - L'alphabet de l'automate est constitué uniquement du caractère c.
4. **Retour de l'automate** :
   - L'automate ainsi construit est retourné. Il reconnaît exactement la chaîne contenant le symbole **c**.

```
automate* creer_automate_symbole(char c) {
    automate* aut = malloc(sizeof(automate));
    aut->nb_etat = 2;
    aut->nb_transition = 1;
    aut->etats = malloc(2 * sizeof(etats));
    aut->transition = malloc(sizeof(transitions));
    aut->alphabet = malloc(26 * sizeof(char));
    aut->etats[0] = (etats){.num_etat = 0, .is_initial = true, .is_final =
false};
```

```
    aut->etats[1] = (etats){.num_etat = 1, .is_initial = false, .is_final =
true};
    aut->transition[0] = (transitions){.etat_depart = aut->etats[0], .etat_arrive
= aut->etats[1], .etiquette = c};

    // Ajouter le symbole à l'alphabet
    aut->alphabet[0] = c;
    aut->nbr_symbole = 1;

    return aut;
}
```

- ✓ **But de la fonction :**

Construire un nouvel automate représentant l'intersection de deux automates déterministes `a1` et `a2`, c'est-à-dire un automate qui accepte uniquement les mots acceptés par les deux automates en même temps.

⇒ **Principe de fonctionnement:**

1. Création de couples d'états :
   - Chaque état du nouvel automate correspond à un couple (e1, e2) d'états des automates a1 et a2.
   - L'état est :
     - initial si e1 est initial et e2 est initial.
     - final si e1 est final et e2 est final.
2. Transitions croisées :
   - Pour chaque paire de transitions ayant le même symbole dans a1 et a2 , une transition est ajoutée dans le nouvel automate entre les états couples correspondants.
3. Alphabet :
   - On suppose que les deux automates utilisent le même alphabet (celui de a2 est copié

- ➢ **Exemple de transformation :**

Si:

- a1 a un état q1 qui va vers q2 avec a.
- a2 a un état p1 qui va vers p2 avec a.

Alors :

- L'intersection aura un état (q1,p1) allant vers (q2,p2) avec le symbole a

```
automate* intersection_automates(automate* a1, automate* a2) {
    int max_etats = a1->nb_etat * a2->nb_etat;
    int max_transitions = a1->nb_transition * a2->nb_transition;
    automate* res = malloc(sizeof(automate));
```

```
    res->etats = malloc(sizeof(etats) * max_etats);
    res->transition = malloc(sizeof(transitions) * max_transitions);
    res->alphabet = a1->alphabet; // On suppose même alphabet
    res->nbr_symbole = a1->nbr_symbole;
    res->nb_etat = 0;
    res->nb_transition = 0;
    // couples d'états
    int etat_map[a1->nb_etat][a2->nb_etat];
    memset(etat_map, -1, sizeof(etat_map));
    for (int i = 0; i < a1->nb_etat; i++) {
        for (int j = 0; j < a2->nb_etat; j++) {
            etat_map[i][j] = res->nb_etat;
            res->etats[res->nb_etat].num_etat = res->nb_etat;
            res->etats[res->nb_etat].is_initial = a1->etats[i].is_initial && a2-
>etats[j].is_initial;
            res->etats[res->nb_etat].is_final = a1->etats[i].is_final && a2-
>etats[j].is_final;
            res->nb_etat++;
        }
    }
    // Pour chaque couple d'états on teste les transitions
    for (int i = 0; i < a1->nb_etat; i++) {
        for (int j = 0; j < a2->nb_etat; j++) {
            for (int x = 0; x < a1->nb_transition; x++) {
                if (a1->transition[x].etat_depart.num_etat != i) continue;
                char symb = a1->transition[x].etiquette;
                // Trouver une transition dans a2 avec même symbole et départ = j
                for (int y = 0; y < a2->nb_transition; y++) {
                    if (a2->transition[y].etat_depart.num_etat == j &&
                        a2->transition[y].etiquette == symb) {

                        int e1_suiv = a1->transition[x].etat_arrive.num_etat;
                        int e2_suiv = a2->transition[y].etat_arrive.num_etat;
                        transitions t;
                        t.etat_depart = res->etats[etat_map[i][j]];
                        t.etat_arrive = res->etats[etat_map[e1_suiv][e2_suiv]];
                        t.etiquette = symb;
                        res->transition[res->nb_transition++] = t;
                    }
                }
            }
        }
    }

    return res;
```

```
}
```

f. Utilitaires:

✓ **But de la fonction :**

La fonction **trouve les états** dans un automate qui ont le plus grand nombre de **transitions sortantes**.

⇒ **Mécanisme de la fonction:**

1. **Calcul du nombre maximal de transitions sortantes** :
   La fonction parcourt tous les états de l'automate et compte leurs transitions sortantes. Elle conserve le **maximum** trouvé.
2. **Identification des états avec le nombre maximal** :
   Ensuite, elle parcourt à nouveau tous les états et les transitions pour **extraire les états** ayant exactement le nombre maximal de transitions sortantes.
3. **Retour** :
   La fonction renvoie un tableau contenant tous les états ayant le plus grand nombre de transitions sortantes et met à jour le nombre d'états max trouvé.

```
// Fonction pour trouver les états avec le plus de transitions sortantes
etats* Etats_Plus_Grand_Transition(automate* automat, int* nb_etats_max) {
    int max = 0;
    *nb_etats_max = 0;

    // Trouver le nombre maximal de transitions sortantes
    for (int i = 0; i < automat->nb_etat; i++) {
        int cpt = 0;
        for (int j = 0; j < automat->nb_transition; j++) {
            if (automat->transition[j].etat_depart.num_etat == automat-
>etats[i].num_etat) {
                cpt++;
            }
        }
        if (cpt > max) {
            max = cpt;
        }
    }

    // Trouver tous les états avec le nombre maximal de transitions sortantes
    etats* etats_max = malloc(automat->nb_etat * sizeof(etats));
    for (int i = 0; i < automat->nb_etat; i++) {
        int cpt = 0;
        for (int j = 0; j < automat->nb_transition; j++) {
```

```
            if (automat->transition[j].etat_depart.num_etat == automat-
>etats[i].num_etat) {
                cpt++;
            }
        }
        if (cpt == max) {
            etats_max[(*nb_etats_max)++] = automat->etats[i];
        }
    }

    return etats_max;
}
```

✓ **But de la fonction :**

La fonction **trouve tous les états** dans un automate qui possèdent une transition **avec une étiquette donnée**.

⇒ **Mécanisme de la fonction :**

1. La fonction **initialise un tableau** pour stocker les états correspondants et un compteur pour suivre le nombre d'états trouvés.
2. Elle parcourt toutes les transitions de l'automate et **compare les étiquettes** des transitions avec l'étiquette donnée.
3. Si une transition correspond, elle ajoute l'état de départ dans le tableau tab_etat et met à jour le nombre d'états trouvés.
4. Enfin, la fonction retourne le tableau d'états qui possèdent une transition avec l'étiquette spécifiée.

```
etats * etat_etiquette (automate *automat, char etiquette, int *nb_etat){
    *nb_etat=0;
    etats * tab_etat = malloc(automat->nb_etat * sizeof(etats));
    int cpt=0;
    for (int i=0; i<automat->nb_etat; i++){
        for (int j=0; j<automat->nb_transition; j++){
            if (automat->transition[j].etat_depart.num_etat == automat-
>etats[i].num_etat && automat->transition[j].etiquette == etiquette){
                tab_etat[cpt]=automat->etats[i];
                cpt++;
                (*nb_etat)++;
            }
        }
    }
    return tab_etat;
}
```

calculer_etats_init(automate* aut)

- **But** : Compter le nombre d'états initiaux dans un automate.
- **Mecanisme** : Parcourt tous les états de l'automate, et incrémente un compteur c pour chaque état marqué comme is_initial.

calculer_etats_fin(automate* aut)

- **But** : Compter le nombre d'états finaux dans un automate.
- **Mecanisme** : Même logique que ci-dessus, mais pour les états ou is_final est vrai.

```
int calculer_etats_init(automate* aut){
    int c=0;
    for (int i = 0; i < aut->nb_etat; i++){
        if(aut->etats[i].is_initial){
            c++;
        }
    }
    return c;
}
int calculer_etats_fin(automate* aut){
    int c=0;
    for (int i = 0; i < aut->nb_etat; i++){
        if(aut->etats[i].is_final){
            c++;
        }
    }
    return c;
}
```

g. Conversion d'expressions régulières:

✓ **But de la fonction**:

Créer un automate simple qui reconnaît un seul caractère c.

⇒ Mecanisme de la fonction

- Crée un automate avec 2 états *:*
  - État 0 : initial
  - État 1 : final
- Une seule transition de l'état 0 vers l'état 1 avec l'étiquette c.
- L'alphabet de l'automate ne contient que le symbole c.

```
automate* creer_automate_symbole(char c) {
    automate* aut = malloc(sizeof(automate));
    aut->nb_etat = 2;
```

```
    aut->nb_transition = 1;
    aut->etats = malloc(2 * sizeof(etats));
    aut->transition = malloc(sizeof(transitions));
    aut->alphabet = malloc(26 * sizeof(char));
    aut->etats[0] = (etats){.num_etat = 0, .is_initial = true, .is_final =
false};
    aut->etats[1] = (etats){.num_etat = 1, .is_initial = false, .is_final =
true};
    aut->transition[0] = (transitions){.etat_depart = aut->etats[0], .etat_arrive
= aut->etats[1], .etiquette = c};

    // Ajouter le symbole à l'alphabet
    aut->alphabet[0] = c;
    aut->nbr_symbole = 1;

    return aut;
}
```

✓ **But de la fonction :**
Analyse récursivement l'expression régulière pour construire progressivement un automate.

⇒ **Mécanisme de la fonction :**

- Parcourrt caractère par caractère l'expression :
  - Crée un automate pour chaque symbole.
  - Applique les opérations : concaténation, union (|), étoile (*), plus (+).
  - Gère les sous-expressions entre parenthèses avec des appels récursifs.
- Retourne l'automate construit au fur et à mesure

```
// Fonction récursive qui analyse l'expression et construit l'automate
automate* regexToAutomateRecursive(const char* regex, int* index) {
    automate* gauche = NULL;

    while (regex[*index] != '\0') {
        char c = regex[*index];
        (*index)++;
        if (isalnum(c)) {
            automate* atom = creer_automate_symbole(c);
            if (gauche == NULL)
                gauche = atom;
            else
                gauche = concatener_automates(gauche, atom);
        }
        else if (c == '.') {
            continue;
        }
        else if (c == '|') {
```

```
            automate* droite = regexToAutomateRecursive(regex, index);
            gauche = union_automate(gauche, droite);
        }
        else if (c == '*') {
            gauche = etoileAutomate(gauche);
        }
        else if (c == '+') {
            gauche = plusAutomate(gauche);
        }
        else if (c == '(') {
            automate* sub = regexToAutomateRecursive(regex, index);
            if (gauche == NULL)
                gauche = sub;
            else
                gauche = concatener_automates(gauche, sub);
        }
        else if (c == ')') {
            return gauche;
        }
    }
    return gauche;
}
automate* regexToAutomate(const char* regex) {
    int index = 0;
    return regexToAutomateRecursive(regex, &index);
}
```

✓ **But de la fonction:**
Fonction principale qui transforme une expression régulière en un automate non déterministe (AFN).

⇒ **Mécanisme de la fonction :**

- Initialise un indice à 0.
- Appelle la fonction récursive regexToAutomateRecursive pour construire l'automate à partir de l'expression régulière.

```
automate* regexToAutomate(const char* regex) {
    int index = 0;
    return regexToAutomateRecursive(regex, &index);
}
```

h. Optimisation et transformations:

⇒ **But de la fonction**

Le but de la fonction calculer_fermeture_epsilon est de déterminer la **ε-fermeture** d'un état donné dans un automate non déterministe.
Autrement dit, elle identifie l'ensemble des états qui peuvent être atteints **à partir d'un état donné**, en suivant **uniquement des transitions epsilon (ε)**, c'est-à-dire des transitions **sans lecture de symbole**.

⇒ **Mécanisme de la fonction**

1. Initialisation de la fermeture :

- L'état de départ est automatiquement inclus dans sa propre fermeture, car un état appartient toujours à sa propre ε-fermeture.

2. Parcours des transitions de l'automate :

- La fonction explore toutes les transitions définies dans l'automate.
- Elle recherche spécifiquement celles qui sont des transitions epsilon, c'est-à-dire des transitions étiquetées par le symbole spécial représentant ε.

3. Détection des états accessibles :

- Lorsqu'une transition epsilon part de l'état courant vers un autre état non encore inclus dans la fermeture, cet état est ajouté à la fermeture.

4. Exploration récursive :

- Pour chaque nouvel état ajouté à la fermeture, la fonction est appelée de manière récursive afin de poursuivre l'exploration à partir de ce nouvel état.
- Cela permet de parcourir en profondeur tous les chemins epsilon disponibles.

Extrait du code :

```
void calculer_fermeture_epsilon(automate *aut, int etat, bool *fermeture) {
    fermeture[etat] = true;
    for (int i = 0; i < aut->nb_transition; i++) {
        if (aut->transition[i].etat_depart.num_etat == etat && aut-
>transition[i].etiquette == 'x') { // ε-transition
            int etat_suivant = aut->transition[i].etat_arrive.num_etat;
            if (!fermeture[etat_suivant]) {
                calculer_fermeture_epsilon(aut, etat_suivant, fermeture);
            }
        }
    }
}
```

⇒ **But de la fonction :**

Le but de la fonction marquer_etats_accessibles est d'identifier et de marquer **tous les états accessibles** à partir d'un **état donné** dans un automate.
Elle permet de déterminer l'ensemble des états que l'on peut atteindre en suivant n'importe quelle **séquence de transitions**, peu importe les symboles de l'alphabet

⇒ **Mécanisme de la fonction :**

1. **Initialisation** :
   L'état de départ est marqué comme accessible dans un tableau booléen. Cela signifie qu'il peut être atteint depuis lui-même.
2. **Exploration des transitions** :
   La fonction parcourt toutes les transitions de l'automate. Elle sélectionne celles qui **partent de l'état courant**.
3. **Découverte des états atteignables** :
   Si une transition mène vers un **état non encore marqué comme accessible**, cet état est ajouté à l'ensemble des états atteints.
4. **Récursivité** :
   Une fois un nouvel état découvert, la fonction s'appelle elle-même de manière récursive pour explorer **les transitions à partir de ce nouvel état**.
5. **Résultat final** :
   À la fin de l'exécution, le tableau booléen contient la **liste complète des états accessibles** à partir de l'état initial.
   Ce traitement est particulièrement utile pour **éliminer les états inaccessibles** ou **optimiser l'automate**.

   Extrait du code :

```
void marquer_etats_accessibles(automate* aut, int etat, bool* accessibles) {
    accessibles[etat] = true;
    for (int i = 0; i < aut->nb_transition; i++) {
        if (aut->transition[i].etat_depart.num_etat == etat && !accessibles[aut-
>transition[i].etat_arrive.num_etat]) {
            marquer_etats_accessibles(aut, aut-
>transition[i].etat_arrive.num_etat, accessibles);
        }
    }
}
```

✓ **But de la fonction :**
Éliminer les transitions spontanées (ε-transitions) pour simplifier l'automate.

⇒ **Mécanisme :**

1. **Calcul des fermetures-ε** pour chaque état (tous les états atteignables via des ε-transitions).
2. **Ajout de transitions directes** pour chaque symbole *a* si un état atteignable via ε a une transition étiquetée *a*.

3. **Marquage des états finaux** si un état dans la fermeture-ε est final.

Extrait du code :

```
automate supprimer_epsilon_transitions(automate aut) {
    bool **fermetures_epsilon = malloc(aut.nb_etat * sizeof(bool *));
    for (int i = 0; i < aut.nb_etat; i++) {
        fermetures_epsilon[i] = calloc(aut.nb_etat, sizeof(bool));
        calculer_fermeture_epsilon(&aut, i, fermetures_epsilon[i]);
    }
    // ... (Mise à jour des transitions et états finaux)
    return new_aut;
}
```

⇒ **But de la fonction :**

La fonction **supprimer_etats_non_accessibles** sert à **éliminer les états non accessibles** d'un automate. Elle crée un **nouvel automate** qui ne conserve que les états et les transitions accessibles à partir des états initiaux, en utilisant un algorithme de parcours.

⇒ **Mécanisme de la function:**

1. **Marquage des états accessibles** :
   La fonction commence par **marquer les états accessibles** en partant des états initiaux à l'aide de la fonction marquer_etats_accessibles.
2. **Comptage des états accessibles** :
   Ensuite, elle compte le nombre d'états marqués comme accessibles.
3. **Création du nouvel automate** :
   Un nouvel automate est alloué avec **seulement les états accessibles**, et les informations relatives aux états, à l'alphabet et aux transitions sont copiées.
4. **Renommage des états** :
   Les états accessibles sont réindexés dans le nouvel automate, les numéros d'état sont mis à jour pour refléter la suppression des états non accessibles.
5. **Copie des transitions** :
   Les transitions de l'automate d'origine sont parcourues. Seules celles reliant deux états accessibles sont copiées dans le nouvel automate, et les numéros des états sont mis à jour pour correspondre à la nouvelle numérotation.
6. **Correction des états initiaux** :
   La fonction garantit qu'il **n'y ait qu'un seul état initial** dans le nouvel automate, en désignant un seul état comme initial et en désactivant les autres.
7. **Libération de la mémoire** :
   Enfin, la mémoire allouée pour le tableau de booléens accessibles est libérée.

```
automate supprimer_etats_non_accessibles(automate* aut) {
    // Marquer les états accessibles
    bool* accessibles = calloc(aut->nb_etat, sizeof(bool));
    for (int i = 0; i < aut->nb_etat; i++) {
        if (aut->etats[i].is_initial) {
            marquer_etats_accessibles(aut, aut->etats[i].num_etat, accessibles);
```

```
        }
    }
    int nb_etats_accessibles = 0;
    for (int i = 0; i < aut->nb_etat; i++) {
        if (accessibles[i]) {
            nb_etats_accessibles++;
        }
    }
    automate* new_aut = malloc(sizeof(automate));
    new_aut->etats = malloc(nb_etats_accessibles * sizeof(etats));
    new_aut->nb_etat = nb_etats_accessibles;
    new_aut->alphabet = malloc(aut->nbr_symbole * sizeof(char));
    memcpy(new_aut->alphabet, aut->alphabet, aut->nbr_symbole * sizeof(char));
    new_aut->nbr_symbole = aut->nbr_symbole;
    int index = 0;
    etats tab_etat[aut->nb_etat][2]; //une table de 2 dimensions contenant les
numéro d'états de l'automate avant et aprés la suppression des états non
accessibles
    for (int i = 0; i < aut->nb_etat; i++) {
        tab_etat[i][0] = aut->etats[i];
        if (accessibles[i]) {
            new_aut->etats[index] = aut->etats[i];
            new_aut->etats[index].num_etat = index;
            tab_etat[i][1] = new_aut->etats[index];
            index++;
        }
    }
    // Copier les transitions des états accessibles
    new_aut->transition = malloc(aut->nb_transition * sizeof(transitions));
    int nb_nouvelles_transitions = 0;
    for (int i = 0; i < aut->nb_transition; i++) {
        if (accessibles[aut->transition[i].etat_depart.num_etat] &&
accessibles[aut->transition[i].etat_arrive.num_etat]) {
            new_aut->transition[nb_nouvelles_transitions] = aut->transition[i];
            new_aut->transition[nb_nouvelles_transitions].etat_depart.num_etat =
aut->transition[i].etat_depart.num_etat;
            new_aut->transition[nb_nouvelles_transitions].etat_arrive.num_etat =
aut->transition[i].etat_arrive.num_etat;
            for(int j=0 ; j<aut->nb_etat; j++){
                if(aut->transition[i].etat_depart.num_etat ==
tab_etat[j][0].num_etat){
                    new_aut-
>transition[nb_nouvelles_transitions].etat_depart.num_etat =
tab_etat[j][1].num_etat;
                }
```

```
                if(aut->transition[i].etat_arrive.num_etat ==
tab_etat[j][0].num_etat){
                    new_aut-
>transition[nb_nouvelles_transitions].etat_arrive.num_etat =
tab_etat[j][1].num_etat;
                }
            }
            nb_nouvelles_transitions++;
        }
    }
    new_aut->nb_transition = nb_nouvelles_transitions;
    // S'assurer qu'un état initial n'est marqué qu'une seule fois
    for (int i = 0; i < new_aut->nb_etat; i++) {
        if (new_aut->etats[i].is_initial) {
            for (int j = i + 1; j < new_aut->nb_etat; j++) {
                if (new_aut->etats[j].is_initial) {
                    new_aut->etats[j].is_initial = false;
                }
            }
        }
    }
    for (int i=0; i<new_aut->nb_etat; i++){
        new_aut->etats[i].num_etat = i;
    }
    // Libérer la mémoire des états accessibles
    free(accessibles);
    return *new_aut;
}
```

✓ **But de la fonction :**
Convertir un NFA (non déterministe) en DFA (déterministe) équivalent.

⇒ **Mécanisme :**

1. **Création d'ensembles d'états** pour représenter les états composites du DFA.
2. **Calcul des transitions** pour chaque symbole de l'alphabet en suivant toutes les transitions possibles du NFA.
3. **Marquage des états finaux** si l'ensemble contient un état final du NFA.

Extrait de code :

```
automate* deterministe_automate(automate* afn) {
    int** ensembles = NULL;
    int nb_ensembles = 0;
    // ... (Gestion des ensembles d'états)
    while (debut < fin) {
        int* courant = file[debut];
        // ... (Calcul des transitions pour chaque symbole)
        for (int s = 0; s < afn->nbr_symbole; s++) {
```

```
            char symb = afn->alphabet[s];
            int prochain[afn->nb_etat * 2];
            // ... (Construction des états suivants)
        }
    }
    return afd;
}
```

✓ **But de la fonction :**
Inverser les transitions de l'automate pour obtenir l'automate "miroir", utilisé dans la minimisation.

⇒ **Mécanisme :**

1. **Inversion des états initiaux et finaux** : Les états finaux deviennent initiaux et vice versa.
2. **Inversion des transitions** : Pour chaque transition $A \rightarrow B$ avec étiquette a, créer $B \rightarrow A$ avec la même étiquette.

Extrait de code :

```
automate* renverser_automate(automate* aut) {
    automate* reversed = malloc(sizeof(automate));
    reversed->etats = malloc(aut->nb_etat * sizeof(etats));
    for (int i = 0; i < aut->nb_etat; i++) {
        reversed->etats[i].is_initial = aut->etats[i].is_final;
        reversed->etats[i].is_final = aut->etats[i].is_initial;
    }
    // ... (Inversion des transitions)
    return reversed;
}
```

i. Interface utilisateur:

- Le **Menu**

```
void menu(){
    int choix;
    automate auto_1;
    automate auto_2;
    automate* resultat;
    do{
        printf("\n------------MENU-------------\n");
        printf("1)-Charger les informations depuis un fichier .dot\n");
        printf("2)-Afficher l'automate\n");
        printf("3)-Concaténer deux automates\n");
        printf("4)-Faire l'union de deux automates\n");
        printf("5)-Faire le plus d'un automate\n");
        printf("6)-Faire l'étoile d'un automate\n");
        printf("7)-Afficher l'etat contenant le plus de transitions\n");
```

```
        printf("8)-Afficher les etats pour une etiquette donnee\n");
        printf("9)-Tester un mot\n");
        printf("10)-Tester les mots dans un fichier texte\n");
        printf("11)-Convertir une expression reguliere en un automate\n");
        printf("12)-Supprimer les epsilon transitions d'un automate\n");
        printf("13)-Faire l'intersection de deux automates\n");
        printf("14)-Menu des automates (initiale, deterministe et minimale)\n");
        printf("15)-Tester les mots dans un fichier texte par automate
minimale\n");
        printf("16)-Quitter\n");
        printf("Entrez votre choix :\n");
        scanf("%d",&choix);
        char nom_fichier[50];
        switch(choix){
            case 1:
            printf("Veuillez saisir le nom du fichier .dot : \n");
            scanf("%s",nom_fichier);
            lire_automate_dot(nom_fichier,&auto_1);
            printf("l'automate est charge avec succes.\n");
            break ;
            case 2 :
            printf("Veuillez saisir le nom du fichier .dot : \n");
            scanf("%s",nom_fichier);
            lire_automate_dot(nom_fichier,&auto_1);
            printf("affichage de l'Automate : \n");
            afficher_automate(&auto_1);
            break ;

//le reste des cases
  }while(choix!=16);
}
```

j. Fonctions annexes:

⇒ **But de la fonction :**

La fonction compare_ints compare deux entiers pour permettre leur tri.

⇒ **Mécanisme de la fonction *:***

Elle prend deux pointeurs vers des entiers, calcule leur différence et renvoie :

- Un nombre négatif si le premier est plus petit,
- Zéro si les deux sont égaux,
- Un nombre positif si le premier est plus grand.

```
// Fonction de comparaison pour le tri
int compare_ints(const void* a, const void* b) {
```

```
    return (*(int*)a - *(int*)b);}
```

⇒ **But de la fonction :**

La fonction contient_etat vérifie si un **état** donné se trouve dans un **ensemble d'états**.

⇒ **Mécanisme de la fonction :**

Elle parcourt l'ensemble des états et retourne true si l'état est trouvé, sinon false après avoir parcouru toute la liste.

```
// Vérifie si un état est dans une liste d'états
bool contient_etat(int* ensemble, int taille, int etat) {
    for (int i = 0; i < taille; i++) {
        if (ensemble[i] == etat) return true;
    }
    return false;
}
```

⇒ **But de la fonction :**

Cette fonction vérifie si **deux ensembles d'états** (représentés par des tableaux d'entiers) sont **égaux**, indépendamment de l'ordre des éléments.

⇒ **Mécanisme de la fonction:**

1. Si les tailles des deux ensembles sont différentes → retourne false.
2. Sinon, elle vérifie que **chaque élément** du premier ensemble est présent dans le second.
3. Si tous les éléments sont trouvés → retourne true, sinon false.

```
// Compare deux ensembles d'états (sans ordre)
bool ensembles_egaux(int* a, int taille_a, int* b, int taille_b) {
    if (taille_a != taille_b) return false;
    for (int i = 0; i < taille_a; i++) {
        if (!contient_etat(b, taille_b, a[i])) return false;
    }
    return true;
}
```

✓ **But de la fonction :**

La fonction cherche à **trouver un ensemble d'états** dans une liste d'ensembles existants ou à **ajouter un nouvel ensemble d'états** si aucun ensemble équivalent n'existe déjà.

⇒ **Mécanisme de la fonction :**

1. La fonction parcourt les ensembles existants et compare chaque ensemble avec le nouvel ensemble d'états.
2. Si un ensemble identique est trouvé, elle retourne son index.
3. Si aucun ensemble équivalent n'est trouvé, elle **ajoute** le nouvel ensemble à la liste, réalloue la mémoire pour l'ajouter à la fin, et met à jour les tailles des ensembles.
4. Retourne l'index du nouvel ensemble ajouté.

```
// Trouve ou crée un état déterministe (ensemble d'états AFN)
```

```
int trouver_ou_ajouter_ensemble(int*** ensembles, int* nb_ensembles, int*
taille_ensemble, int* nouvel_ensemble, int taille_nouvel) {
    for (int i = 0; i < *nb_ensembles; i++) {
        if (ensembles_egaux((*ensembles)[i], taille_ensemble[i], nouvel_ensemble,
taille_nouvel)) {
            return i; // déjà existant
        }
    }

    // Ajouter nouveau
    (*ensembles) = realloc(*ensembles, (*nb_ensembles + 1) * sizeof(int*));
    (*ensembles)[*nb_ensembles] = malloc(taille_nouvel * sizeof(int));
    memcpy((*ensembles)[*nb_ensembles], nouvel_ensemble, taille_nouvel *
sizeof(int));

    taille_ensemble[*nb_ensembles] = taille_nouvel;
    return (*nb_ensembles)++;
}
```

III. Guide d'utilisation

1. Introduction

Un **automate fini** ou **automate avec un nombre fini d'états** est un modèle mathématique de calcul, utilisé dans de nombreuses circonstances, allant de la conception de programmes informatiques et de circuits en logique séquentielle aux applications dans des protocoles de communication, en passant par le contrôle des processus, la linguistique et même la biologie. Un automate fini est une construction mathématique abstraite, susceptible d'être dans un nombre fini d'*états*, mais étant un moment donné dans un seul état à la fois ; l'état dans lequel il se trouve alors est appelé l'« **état courant** ». Le passage d'un état à un autre est activé par un événement ou une condition ; ce passage est appelé une « **transition** ». Un automate particulier est défini par l'ensemble de ses états et l'ensemble de ses transitions.
Un exemple d'un automate fini de 2 états (0 et 1) qui accepte les mots ayant la première lettre est « a » suivi par 0, 1 ou plusieurs « b » :

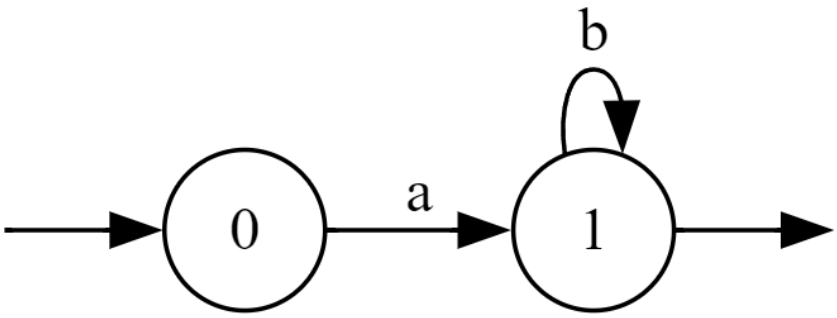


Figure 1: Exemple d'un automate fini simple

Dans ce projet, nous avons développé un programme en langage C permettant de lire des automates depuis des fichiers .dot et vérifier les mots acceptées …, ainsi que des opérations tels que la concaténation, union, détermination, minimalisation … Visé aux passionnés d'informatique théorique, ce projet offre une

interface en ligne de commande (cmd) avec un menu simple et efficace pour comprendre pour bien comprendre et expérimenter le fonctionnement des automates finis. Ce rapport d'utilisation, nous expliquerons comment exécuter le programme, comment fournir les fichiers d'entrée nécessaires, et comment interpréter les résultats afficher les résultats.

2. Installation et exécution

i) Prérequis

Avant d'utiliser le programme, assurez-vous que votre environnement dispose des éléments suivants :

- Un éditeur de texte (IDE) : pour écrire le code source du programme et pour la modification des fichiers d'entré. En théorie un logiciel comme bloc-notes ou VS Code fait l'affaire.
- Un compilateur comme **gcc** pour compiler le code
- Une ligne de commande (Windows) ou bien un terminal (LINUX)

ii) Compilation et exécution

A l'aide du IDE choisis, vous pouvez exécuter le code en cliquant sur (Run code)

3. Format des fichiers d'Entrée

i) Fichier .dot

Le fichier d'entré qui contient l'automate a traiter doit être un fichier d'extension .dot écrit avec le langage graphviz et doit respecter les critère suivants :

- ***Les deux premières lignes***

Un fichier .dot doit commencer par digraph G dans la première ligne, suivi par **rankdir = LR** (qui signifie qu'il apparait de gauche à droite).

- ***Déclaration des états***

Puis dans la troisième ligne on déclare tous les états de notre automate, en suivant cette structure :
node [shape = circle] ; 0, 1, 2, …, n; (avec n nombre d'états -1)

- ***Etat initiale***

On déclare un point invisible (cercle de diamètre = 0) comme ça node [shape = point width = 0] ; start1;
Puis on ajoute une flèche sortante de ce point invisible vers l'état initiale : **start1 -> «etat_initiale »;**

- ***Etat finale***

On suit presque la même procédure que l'état initial, on créer le point invisible **node [shape = point width = 0] ; start1;** puis on ajoute une flèche sortante de l'état final vers le point invisible : **« etat_fin » -> final1;**

- ***Remarque concernant les Etats initiaux et finaux :***

On commence par start1 ou final1 puis on augmente le nombre selon le nombre des états initiaux et finaux dans l'automate (start2, start3 …).

- ***Transitions***

Ensuite on déclare les transitions par la structure suivant **« etat_depart » -> « etat_arrivé » [label= « a »] ;** (attention il faut respecter les espaces). Enfin on ferme l'accolade «}» pour marquer la fin de l'automate.

ii) Fichier .txt

Les fichiers .txt qui contiennent les mots à tester n'ont pas un format spécifique, vous devez juste saisir les mots à tester dans un fichier txt, séparer chaque 2 mots par un espace ou bien saute de ligne

iii) La ligne de commande

Dans notre menu il y a des fonctions qui demande de l'utilisateur de saisir une chaine de caractère (expression régulière)
Donc pour la structure de cette chaine de caractère, il faut respecter les critères suivants :

- N'utilisez pas les espaces
- L'union est représentée par « | »
- Pour faire étoile ou plus il suffit d'écrire « * » ou « + » sans les précédé par « ^ ».

4. Interprétation des résultats

Une fois le programme est exécuté, il affichera le menu suivant et attend que l'utilisateur choisit une option par saisir le numéro correspondant :

```
-----------MENU-------------
1)-Charger les informations depuis un fichier .dot
2)-Afficher l'automate
3)-Concat├®ner deux automates
4)-Faire l'union de deux automates
5)-Faire le plus d'un automate
6)-Faire l'├®toile d'un automate
7)-Afficher l'etat contenant le plus de transitions
8)-Afficher les etats pour une etiquette donnee
9)-Tester un mot
10)-Tester les mots dans un fichier texte
11)-Convertir une expression reguliere en un automate
12)-Supprimer les epsilon transitions d'un automate
13)-Faire l'intersection de deux automates
14)-Menu des automates (initiale, deterministe et minimale)
15)-Tester les mots dans un fichier texte par automate minimale
16)-Quitter
Entrez votre choix :
```

Figure 2: Menu principale de programme

Après choisir une option, le programme doit demander les données d'entrée (les fichiers contenant les automates ou les fichiers .txt pour tester les mots ou une chaine de caractères écrit dans la ligne de commande)

Option n°1 :

Pour charger l'automate écrit dans un fichier.dot pour l'utiliser après :

```
Entrez votre choix :
1
Veuillez saisir le nom du fichier .dot :
aut1.dot
l'automate est charge avec succes.
```

Saisir le nom du fichier + extension .dot à la fin

Option n°2 :

Pour afficher les informations de l'automate chargé dans un fichier .dot

```
Entrez votre choix :
2
Veuillez saisir le nom du fichier .dot :
aut2.dot
affichage de l'Automate :
Nombre d'etats : 2
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Oui

Nombre de transitions : 2
0 --a--> 1
1 --b--> 1
Les alphabets de l'automate sont : {a,b}
```

Affichage des différentes informations concernant l'automate contenu dans le fichier.dot saisi

Option n°3 :

Option de concaténer deux automates :

- Le programme demande à l'utilisateur de donner les fichiers qui contient les automates à concaténer
- Puis affiche le résultat et enfin demande à l'utilisateur de saisir le nom de fichier ou il doit sauvegarder le nouvel automate

```
Entrez votre choix :
3
Veuillez saisir le nom du premier fichier .dot :
aut1.dot
Veuillez saisir le nom du deuxi├¿me fichier .dot :
aut2.dot
Les automates ont ├®t├® concat├®n├®s avec succ├¿s.
Nombre d'etats : 4
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Non
Etat 2, Initial: Non, Final: Non
Etat 3, Initial: Non, Final: Oui

Nombre de transitions : 5
0 --a--> 1
1 --b--> 0
2 --a--> 3
3 --b--> 3
1 --x--> 2
Les alphabets de l'automate sont : {a,b}
Veuillez saisir un nom pur le fichier que vous souhaitez Creer: conca.dot
```

L'utilisateur saisit "3" qui corresspend à "concatener deux automates" dans le menu

Premièr fichier d'entré

deuxièm fichier d'entré

Les informations de l'automate résultante aprés concaténation

Nom de fichier ou on stocke l'automate résultante

Automate initial :

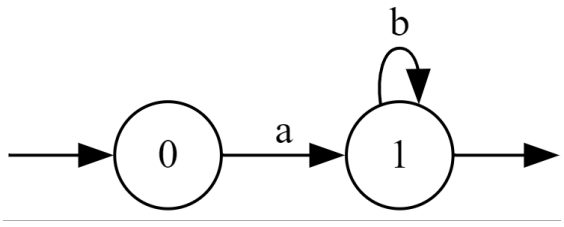


Figure 3: automate initiale (aut_1)

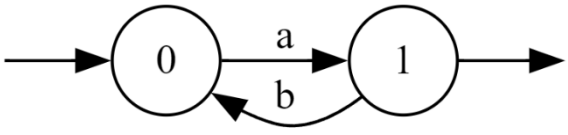


Figure 3:automate initiale (aut 2)

Automate résultant :

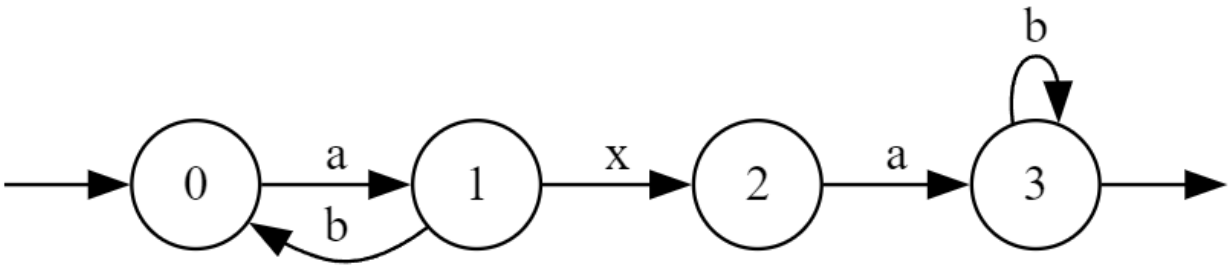


Figure 4: résultat de concaténation

Option n°4 :

Du même pour l'union de deux automates.

```
Entrez votre choix :
4
Veuillez saisir le nom du premier fichier .dot :
aut1.dot
Veuillez saisir le nom du deuxi├¿me fichier .dot :
aut2.dot
Les automates ont ├®t├® unis avec succ├¿s.
Nombre d'etats : 6
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Non
Etat 2, Initial: Non, Final: Non
Etat 3, Initial: Non, Final: Non
Etat 4, Initial: Non, Final: Non
Etat 5, Initial: Non, Final: Oui

Nombre de transitions : 8
1 --a--> 2
2 --b--> 1
3 --a--> 4
4 --b--> 4
0 --x--> 1
0 --x--> 3
2 --x--> 5
4 --x--> 5
Les alphabets de l'automate sont : {a,b}
Veuillez saisir un nom pur le fichier que vous souhaitez Creer: resultat_union.dot
```

Saisir le nom des deux fichiers à unir avec l'extension .dot

Message de confirmation est affiché.
Puis, les informations de l'automate résultant sont

L'opération est cloture par choisir un nom pour le fichier.dot qui contiendra l'automate résultant

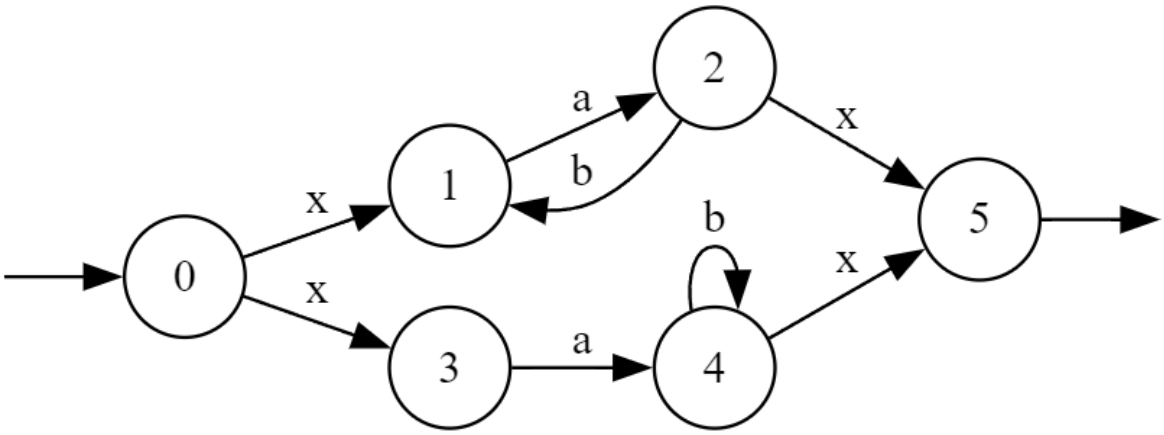


Figure 5: automate après union de deux automates

Option n°5 :

Opération "Plus" (+) : Répétition une ou plusieurs fois

- ➔ L'automate A+ accepte les mots formés par **une ou plusieurs** répétitions consécutives d'un mot reconnu par A.

Du même, on procède par la saisie du nom du fichier.dot qu'on y applique l'opération, ainsi que le nom du fichier.dot de l'automate résultant.

```
Entrez votre choix :
5
Veuillez saisir le nom du fichier .dot :
aut1.dot
Le plus de l'automate a ├®t├® cr├®├®e avec succ├¿s.
Nombre d'etats : 2
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Oui

Nombre de transitions : 3
0 --a--> 1
1 --b--> 0
1 --x--> 0
Les alphabets de l'automate sont : {a,b}
Veuillez saisir un nom pur le fichier que vous souhaitez Creer: resultat_plus.dot
```

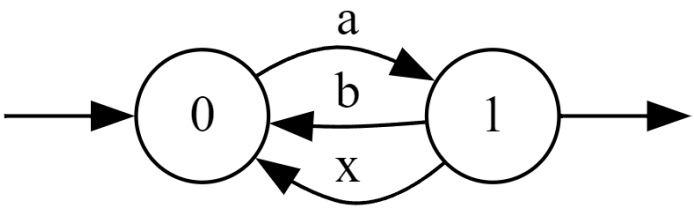


Figure 6 : Résultat de plus d'un automate

Option n°6 :

Opération "Étoile de Kleene" (*) : Répétition zéro ou plusieurs fois.
L'automate A* accepte :

- Le **mot vide** (ε).
- Tous les mots formés par **zéro ou plusieurs** répétitions consécutives d'un mot reconnu par A.

```
Entrez votre choix :
6
Veuillez saisir le nom du fichier .dot :
aut1.dot
L'├®toile de l'automate a ├®t├® cr├®├®e avec succ├¿s.
Nombre d'etats : 4
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Non
Etat 2, Initial: Non, Final: Non
Etat 3, Initial: Non, Final: Oui

Nombre de transitions : 6
1 --a--> 2
2 --b--> 1
2 --x--> 1
0 --x--> 3
0 --x--> 1
2 --x--> 3
Les alphabets de l'automate sont : {a,b}
Veuillez saisir un nom pur le fichier que vous souhaitez Creer: resultat_etoile.dot
```

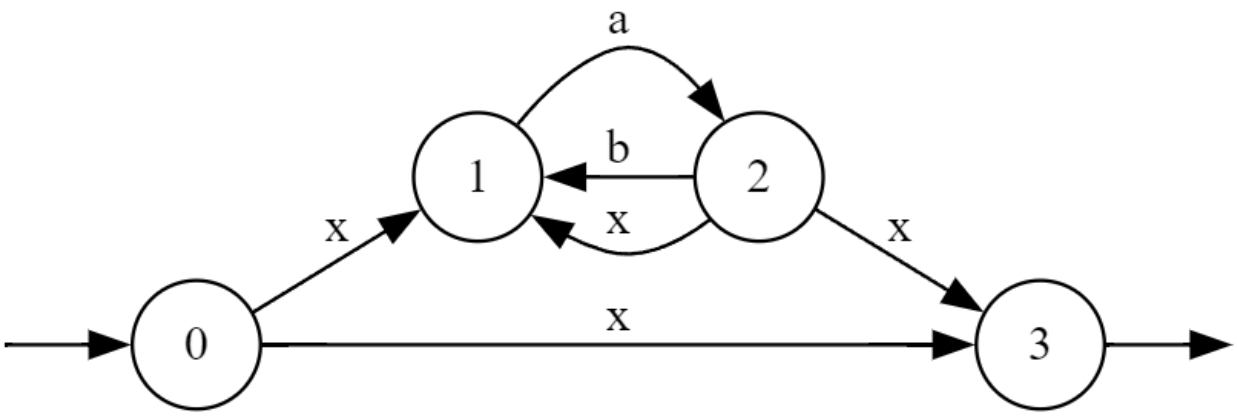


Figure 7: Résultat d'étoile d'un automate

Automate initiale pour tester :

Cet automate est utilisé pour tester dans les options 7, 8 et 9

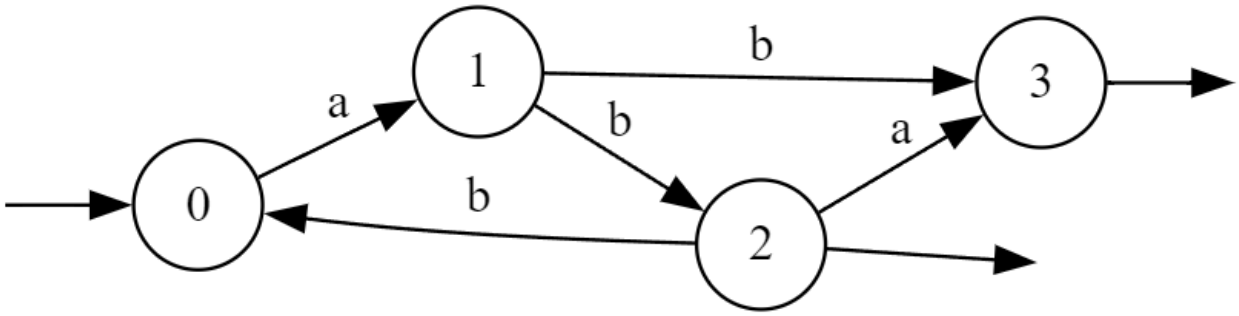


Figure 8: automate de test

Option n°7 :

Cette option sert à afficher les états qui contient les plus de transitions sortantes dans un automate :
On a comme résultat état 1 et état 2 contiennent les plus de transitions sortantes.

```
Entrez votre choix :
7
Veuillez saisir le nom du fichier .dot :
test5.dot
Les etats contenant le plus de transitions sont :
Etat 1
Etat 2
```

Option n°8 :

Cette option affiche les états avec une transition sortante étiqueté par un caractère saisi.

```
Entrez votre choix :
8
Veuillez saisir le nom du fichier .dot :
test5.dot
Entrez l'etiquette : a
Les etats avec l'etiquette 'a' sont :
Etat 0
Etat 2
```

Entrer l'étiquette que vous souhaitez savoir ses états de départ.

Les états **0** et **2** admettent des transitions sortantes étiquetées par **a**

Option n°9 :

Cette option sert à tester si un mot est accepté par un automate déjà chargé, en saisissant le mot à tester.
Puis le programme affiche un message de confirmation (mot accepté ou non).

```
Entrez votre choix :
9
Entrez un mot a tester : aba
Le mot "aba" est accepte par l'automate.
```

Option n°10 :

L'opération sert à tester des mots contenus dans un fichier.txt s'ils sont acceptee par l'automate chargee ou non , en saisissant le nom du fichier.txt contenant les mots à tester.

```
Entrez votre choix :
10
Veuillez saisir le nom du fichier texte :
test.txt
Le mot "aab" est rejete par l'automate.
Le mot "ab" est accepte par l'automate.
Le mot "ba" est rejete par l'automate.
Le mot "abb" est rejete par l'automate.
Le mot "abb" est rejete par l'automate.
Le mot "aba" est accepte par l'automate.
```

Option n°11 :

Cette option sert à transformer une expression régulière à un automate. On commence par saisir l'expression régulière, puis le programme afficher les informations du nouvel automate.

```
Entrez votre choix :
11
Entrez l'expression r├®guli├¿re : (a|b)+b*
L'automate a ├®t├® cr├®├® avec succ├¿s.
Nombre d'etats : 10
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Non
Etat 2, Initial: Non, Final: Non
Etat 3, Initial: Non, Final: Non
Etat 4, Initial: Non, Final: Non
Etat 5, Initial: Non, Final: Non
Etat 6, Initial: Non, Final: Non
Etat 7, Initial: Non, Final: Non
Etat 8, Initial: Non, Final: Non
Etat 9, Initial: Non, Final: Oui

Nombre de transitions : 13
2 --a--> 3
4 --b--> 5
1 --x--> 2
1 --x--> 4
3 --x--> 6
5 --x--> 6
6 --x--> 1
7 --b--> 8
6 --x--> 7
8 --x--> 1
0 --x--> 9
0 --x--> 1
8 --x--> 9
Les alphabets de l'automate sont : {a,b}
Veuillez saisir un nom pour le fichier que vous souhaitez Creer : resultat_reg.dot
```

Saisir l'expression régulière à transformer en automate

Nom du nouveau fichier.dot contenant le résultat

Automate résultant :

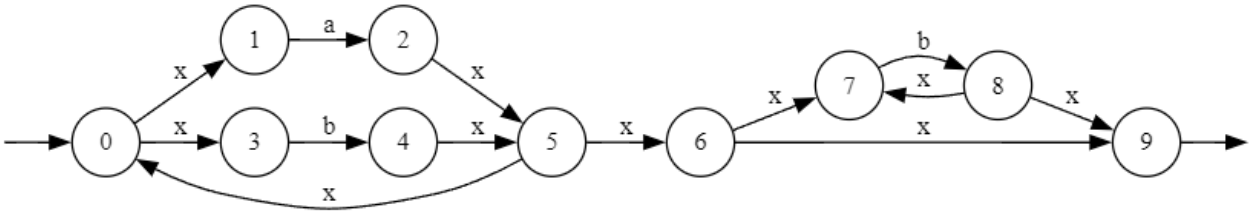


Figure 9 : automate résultant de l 'expression régulière saisie

Option n°12 :

Pour supprimer les epsilon transition (les epsilons sont représentés par des 'x') :

- Choisir l'option 12
- Saisir le nom du fichier.dot que vous souhaitez d'y supprimer les epsilons transitions
- Saisir un nom de fichier.dot pour y inclure le résultat de la suppression
- Le programme affichera par la suite les éléments de votre automate résultant
- Finalement, vous pouvez consulter votre fichier.dot parmi les fichiers existant dans votre environnement

```
Entrez votre choix :
12
Veuillez saisir le nom du fichier .dot qui contient les epsilon transitions :
test_epsilon.dot
Veuillez saisir un nom pour le fichier que vous souhaitez Creer: resultat_epsilon.dot
Nombre d'etats : 2
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Oui

Nombre de transitions : 2
0 --a--> 1
0 --b--> 1
Les alphabets de l'automate sont : {a,b}
```

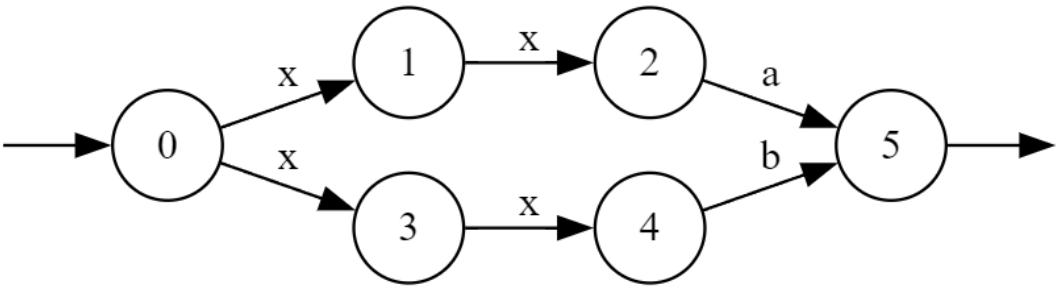


Figure 10: Automate avant suppression des epsilon transitions

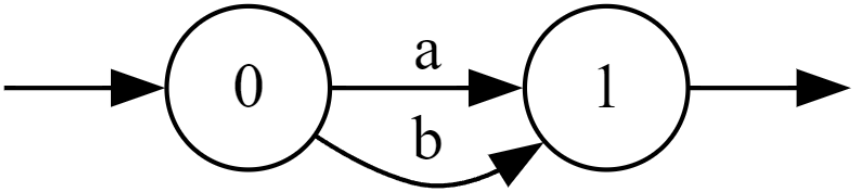


Figure 11: Automate après suppression des epsilon transitions

Option n°13 :

L'opération suivante sert à faire l'intersection de deux automates contenus dans deux fichier.dot saisis par l'utilisateur , qui choisi en fin le nom du fichier.dot contenant l'automate résultant.

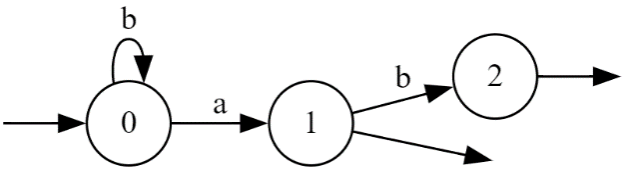


Figure 12: Automate 2 initial d'intersection

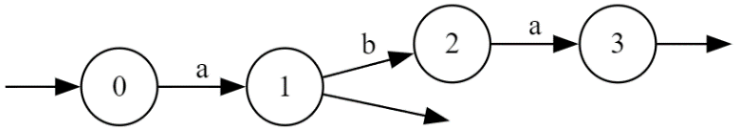


Figure 13: automate initial d'intersection

```
Entrez votre choix :
13
Veuillez saisir le nom du premier fichier .dot :
test_intersection_1.dot
Veuillez saisir le nom du deuxi├¿me fichier .dot :
test_intersection_2.dot
Veuillez saisir le nom du fichier .dot de destination :
resultat_intersection.dot
Nombre d'etats : 6
Etats :
Etat 0, Initial: Oui, Final: Non
Etat 1, Initial: Non, Final: Non
Etat 2, Initial: Non, Final: Non
Etat 3, Initial: Non, Final: Non
Etat 4, Initial: Non, Final: Non
Etat 5, Initial: Non, Final: Oui

Nombre de transitions : 12
0 --a--> 3
0 --b--> 1
1 --a--> 2
1 --b--> 0
2 --b--> 5
2 --a--> 3
3 --b--> 4
3 --a--> 2
4 --a--> 3
4 --b--> 1
5 --a--> 2
5 --b--> 0
Les alphabets de l'automate sont : {a,b}
```

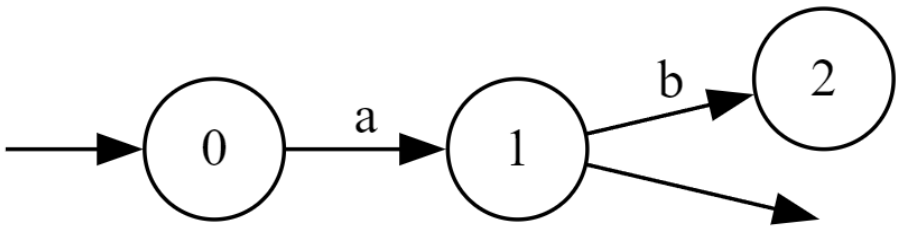


Figure 14: Résultat d'intersection

Option 14 :

L'option suivante nous mène vers un autre menu où l'utilisateur peut choisir un choix valable du menu, puis saisir le nom du fichier.dot souhaité.
En fin , il va choisir un nom pour le fichier.dot de l'automate résultant.

a. Générer automate initial:

```
Entrez votre choix :
14
Veuillez saisir le nom du fichier .dot initiale :
union1.dot

-----------MENU--------------
1)-Generer automate initiale
2)-generer automate deterministe
3)-Generer automate minimale
4)-Quitter
Entrez votre choix :
1
Veuillez saisir le nom du fichier .dot de l'automate initiale ├á creer :
testinitial1.dot
L'automate initiale a ├®t├® cr├®├® avec succ├¿s.
```

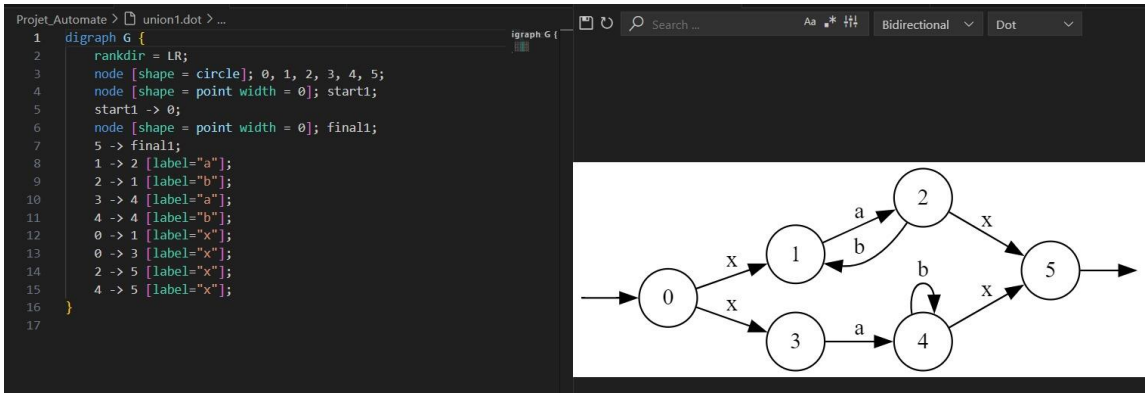


Figure 15 : automate initial du test

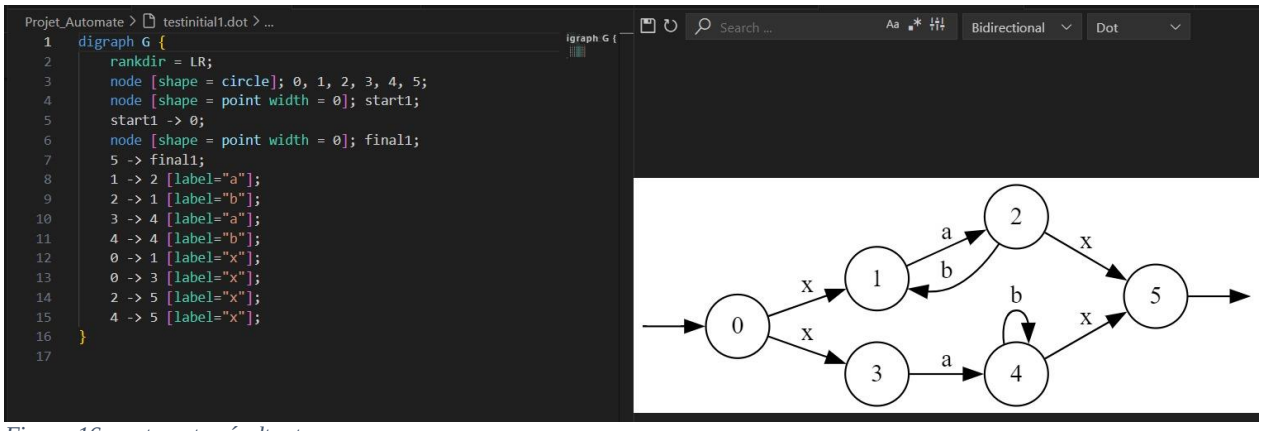


Figure 16 : automate résultant

b. Générer automate déterministe :

```
Entrez votre choix :
14
Veuillez saisir le nom du fichier .dot initiale :
aut1.dot

------------MENU--------------
1)-Generer automate initiale
2)-generer automate deterministe
3)-Generer automate minimale
4)-Quitter
Entrez votre choix :
2
Veuillez saisir un nom pour le fichier que vous souhaitez Creer: Deter.dot
```

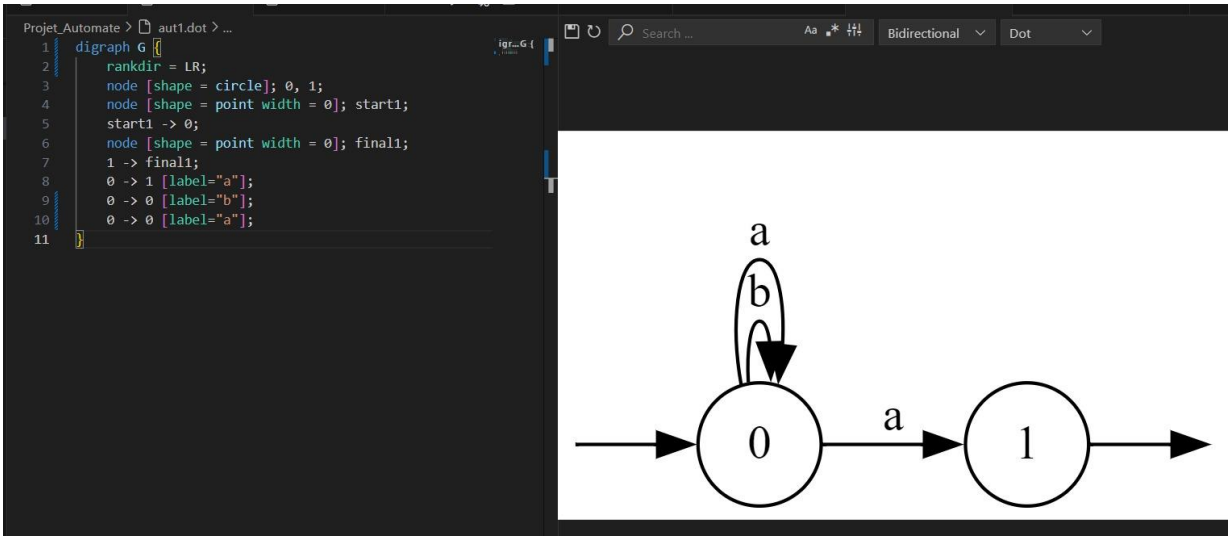


Figure 17 : automate initial du test de determinisation

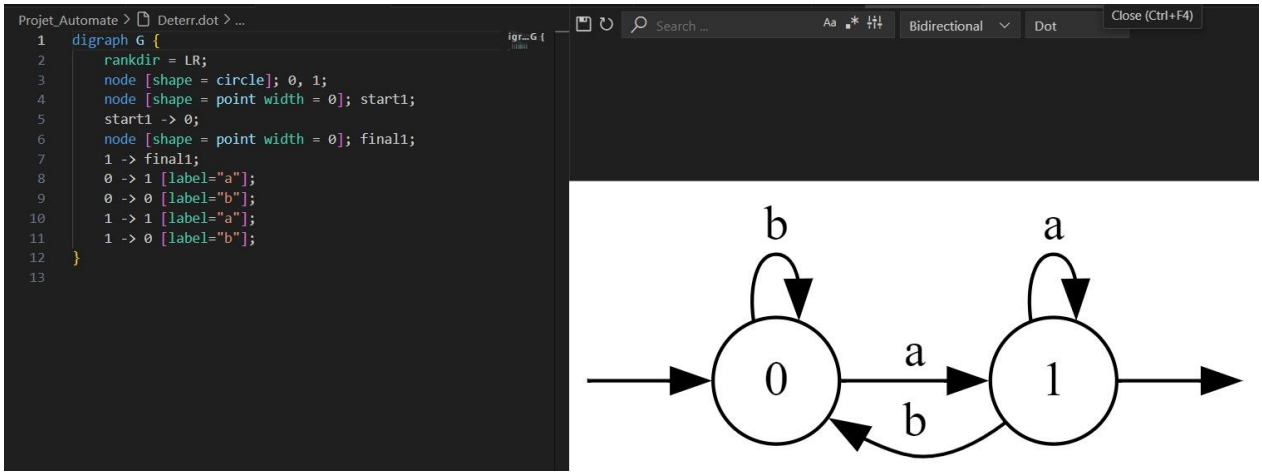


Figure 18 : résultat du déterminisation

c. Générer automate minimal :

```
3)-Generer automate minimale
4)-Quitter
Entrez votre choix :
3
Veuillez saisir le nom du fichier .dot á minimiser :
test5.dot
Veuillez saisir le nom du fichier .dot á creer :
minimisationn.dot
```

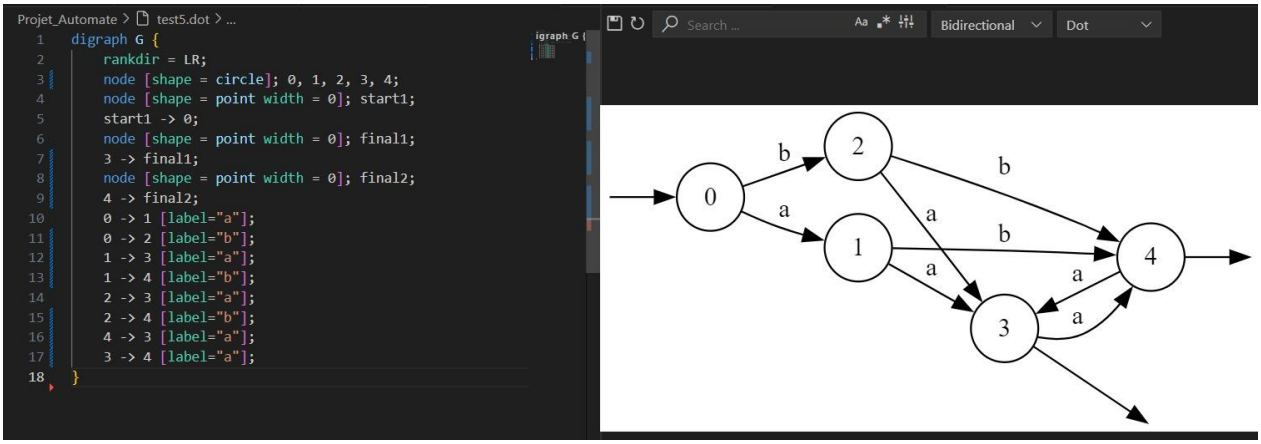


Figure 19 : automate à minimiser

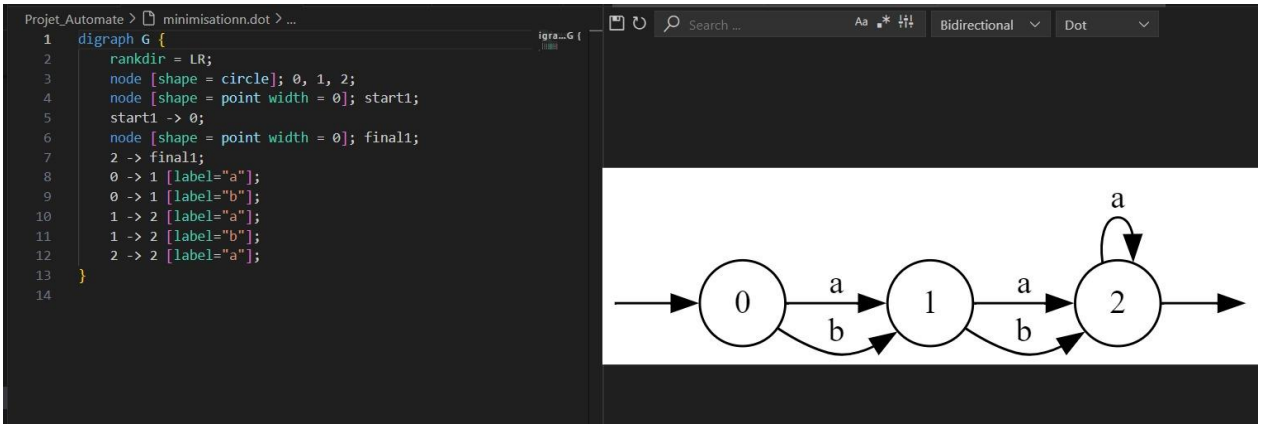


Figure 20 : automate résultant

➔ Le 4em choix permet de revenir au menu initial de notre programme .

```
------------MENU--------------
1)-Generer automate initiale
2)-generer automate deterministe
3)-Generer automate minimale
4)-Quitter
Entrez votre choix :
4
Au revoir !
```

Option 15 :

Cette option permet de tester des mots contenus dans un fichier.txt s'ils sont acceptés pqr lùquto,qte chargé actuellement ou non.

```
Entrez votre choix :
15
Veuillez saisir le nom du fichier .dot de l'automate ├á test├® :
minimisationn.dot
Veuillez saisir le nom du fichier texte :
test.txt
Le mot "aab" est rejete par l'automate.
Le mot "ab" est accepte par l'automate.
Le mot "ba" est accepte par l'automate.
Le mot "abb" est rejete par l'automate.
Le mot "abb" est rejete par l'automate.
```

Le programme affiche les mots en précisant s'ils sont acceptés ou non.

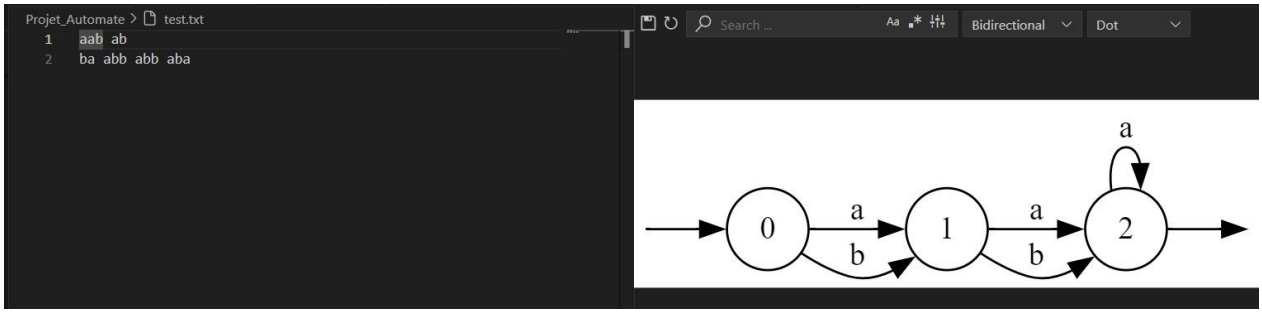


Figure 21 : l'automate du test et le fichier.txt contenant les mots à tester

Option 16 :

Cette option illustre la fin de notre programme où l'utilsateur a bien profité des différentes options assurées de la part de notre projet.

